DeepDumps

Premium exam material

Get certification quickly with the **Deep Dumps** Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily.

https://www.DeepDumps.com

About DeepDumps

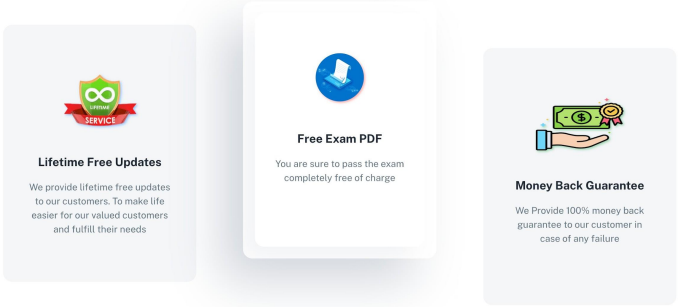
At DeepDumps, we finally had enough of the overpriced, paywall-ridden exam prep industry. Our team of experienced IT professionals spent years navigating the certification landscape, only to realize that most prep materials came with a price tag bigger than the exam itself. We saw aspiring professionals spending more on study guides than on rent, and frankly, it was ridiculous. So, we decided to change the game. DeepDumps was born with one goal: to provide high-quality exam preparation without draining your wallet.
With DeepDumps, you get expert-vetted study materials, real-world insights, and a fair shot at success—without the financial headache..

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://DeepDumps.com

Mail us on - support@deepdumps.com



John

Thank you so much for your help. I scored 972 in my exam today. More than 30 questions were there from your PDFs!

Passed my exam today with 845 marks

Passed my exam today with 845 marks! Out of 52 questions, 51 were from your DeepDumps PDFs including Contoso case study. I will recommend you to my friends. Thank you DeepDumps team!

Dana

Thanks a lot for this updated AZ-900 Q&A. I just finished my exam and got 374. I followed both of your AZ-900 videos and the 6 PDF, the PDFs are very accurate, all answers are correct. Could you please create a similar video/PDF for DP900, your content-/PDF's is really awesome. The tem'd a really good job. Thank You

These questions are real and 100 % valid.

Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Ahamod Shibly

Customer support is realy fast and friendly, I just finished with me exam I passed it along with the 6 PDF's you

Esmaria

Simple, easy to understand explanations. To anyone whio is writing the exam of AZ900,

Databricks

(Certified Data Engineer Professional)

Certified Data Engineer Professional

Total: **369 Questions**

Link: https://deepdumps.com/papers/databricks/certified-data-engineer-professional

Question: 1 **Deep Dumps**

An upstream system has been configured to pass the date for a given batch of data to the Databricks Jobs API as a parameter. The notebook to be scheduled will use this parameter to load data with the following code: df = spark.read.format("parquet").load(f"/mnt/source/(date)")
Which code block should be used to create the date Python variable used in the above code block?

A.date = spark.conf.get("date")

B.input_dict = input()
date= input_dict["date"]

C.import sys
date = sys.argv[1]

D.date = dbutils.notebooks.getParam("date")

E.dbutils.widgets.text("date", "null")
date = dbutils.widgets.get("date")

Answer: E

Explanation:

The correct answer is **E**, which uses Databricks widgets to handle the date parameter passed from the Jobs API. Let's break down why:

1. **Databricks Widgets:** Widgets are UI elements that allow users to pass parameters to notebooks at runtime. This is crucial for scheduled jobs that need dynamic input. dbutils.widgets.text("date", "null") creates a text widget named "date" with a default value of "null". dbutils.widgets.get("date") retrieves the value of this widget. When the Databricks Jobs API calls the notebook, it can override the default value with the batch date.
2. **Why other options are incorrect:**
3. **A. date = spark.conf.get("date")**: spark.conf.get() retrieves Spark configuration properties, not parameters passed directly to a notebook job. Spark configuration settings are typically for Spark engine behaviors and resources.
4. **B. input_dict = input() date= input_dict["date"]**: This approach expects user input via the console, which is unsuitable for automated, scheduled jobs. Jobs API does not provide console input to scheduled tasks.
5. **C. import sys date = sys.argv[1]**: This is suitable for command-line scripts but not Databricks notebooks invoked by the Jobs API. While sys.argv can pass arguments to a Python script, the Databricks Jobs API is not designed to directly populate sys.argv when executing notebooks. The Jobs API parameter passing mechanism is integrated with Databricks dbutils.
6. **D. date = dbutils.notebooks.getParam("date")**: dbutils.notebooks.getParam() is intended for passing parameters between notebooks when using the %run magic command or dbutils.notebook.run(), not for receiving parameters from the Jobs API. It is designed for notebook workflows, not external API calls.
7. **Benefits of using widgets:**
8. **Parameterization:** Widgets enable you to parameterize your notebooks, making them reusable for different data batches or scenarios.
9. **Automation:** The Jobs API can set widget values when scheduling notebooks, enabling automated data processing pipelines.
10. **User Interface:** Widgets are visible and editable in the Databricks notebook UI, which is useful for debugging and interactive exploration.

In summary, using Databricks widgets is the correct and most efficient approach for retrieving parameters passed to a notebook by the Databricks Jobs API, allowing for automated, parameterized data processing workflows.

Authoritative Links:

1. **Databricks Widgets:** https://docs.databricks.com/notebooks/widgets.html
2. **Databricks Jobs API:** https://docs.databricks.com/api/workspace/jobs
3. **dbutils.notebook.run():** https://docs.databricks.com/notebooks/notebook-workflows.html#run-a-notebook

Question: 2 **Deep Dumps**

The Databricks workspace administrator has configured interactive clusters for each of the data engineering groups. To control costs, clusters are set to terminate after 30 minutes of inactivity. Each user should be able to execute workloads against their assigned clusters at any time of the day.
Assuming users have been added to a workspace but not granted any permissions, which of the following describes the minimal permissions a user would need to start and attach to an already configured cluster.

A."Can Manage" privileges on the required cluster

B.Workspace Admin privileges, cluster creation allowed, "Can Attach To" privileges on the required cluster

C.Cluster creation allowed, "Can Attach To" privileges on the required cluster

D."Can Restart" privileges on the required cluster

E.Cluster creation allowed, "Can Restart" privileges on the required cluster

Answer: D

Explanation:

Here's a detailed justification for why option D ("Can Restart" privileges on the required cluster) is the most appropriate answer, along with supporting concepts and authoritative links:

The core requirement is to allow users to execute workloads against already configured clusters that might be terminated due to inactivity. The key here is that the clusters already exist. We are not creating new clusters. Therefore, any option requiring cluster creation privileges can be immediately eliminated (options B, C, and E).

"Can Manage" privileges (option A) are far too broad. "Can Manage" grants the user full control over the cluster, including editing its configuration, resizing it, deleting it, and managing permissions. The question asks for minimal permissions. "Can Manage" goes beyond the scope of simply starting and attaching.

The Databricks documentation defines specific cluster access control levels. A cluster terminated due to inactivity does not necessarily need to be completely re-created (requiring "Can Manage"). Instead, it can be restarted.

When a cluster terminates due to inactivity, it enters a terminated state, but the cluster configuration persists. A user with "Can Restart" privileges can bring the cluster back online using its existing configuration. Once the cluster is running, they can then attach to it to execute workloads.

"Can Attach To" is inherently required to execute workloads against a running cluster. However, if the cluster is terminated, "Can Attach To" alone is insufficient; the cluster must first be restarted. Since the cluster is configured to terminate after inactivity, the ability to restart becomes crucial.

Therefore, the combination of "Can Restart" allows the user to bring the stopped cluster back up, and implicitly the ability to "Can Attach To" allows the user to execute workloads when the cluster is running. The minimal permission required is "Can Restart" because, in the scenario described, the user needs to be able to bring the cluster from a terminated (inactive) state to a running state, then "Can Attach To" access is given automatically.

Authoritative Links:

Databricks Cluster Access Control: https://docs.databricks.com/security/access-control/cluster-access-control.html
Databricks Cluster Lifecycle: https://docs.databricks.com/clusters/clusters-manage.html#cluster-states

Question: 3 **Deep Dumps**

When scheduling Structured Streaming jobs for production, which configuration automatically recovers from query failures and keeps costs low?

A.Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: Unlimited

B.Cluster: New Job Cluster;
Retries: None;
Maximum Concurrent Runs: 1

C.Cluster: Existing All-Purpose Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1

D.Cluster: New Job Cluster;
Retries: Unlimited;
Maximum Concurrent Runs: 1

E.Cluster: Existing All-Purpose Cluster;
Retries: None;
Maximum Concurrent Runs: 1

Answer: D

Explanation:

Here's a detailed justification for why option D (Cluster: New Job Cluster; Retries: Unlimited; Maximum Concurrent Runs: 1) is the most appropriate choice for scheduling production Structured Streaming jobs on Databricks, emphasizing fault tolerance and cost efficiency:

The key requirements for production Structured Streaming jobs are reliability (automatic recovery from failures) and cost optimization. A "New Job Cluster" is purpose-built for a specific task, which ensures resource isolation. This means that a failure in one streaming job will not impact other jobs. Creating a new cluster for each job provides a clean environment.

"Retries: Unlimited" provides the desired fault tolerance. If a streaming query fails due to transient issues (network glitch, temporary resource unavailability), Databricks will automatically restart it. Without retries, a failure would require manual intervention, leading to data loss or processing delays.

"Maximum Concurrent Runs: 1" enforces sequential execution. While running multiple concurrent jobs on the same cluster might seem appealing for resource utilization, it introduces complexity regarding resource contention and potential interference. Limiting to one run at a time simplifies management and minimizes the risk of failures due to resource exhaustion.

Option A is incorrect due to "Maximum Concurrent Runs: Unlimited." Running unlimited jobs concurrently could overload the cluster, leading to instability and increased failure rates. Option B is flawed because "Retries: None" means the job fails permanently upon encountering an error. Option C is not a good choice due to using an "Existing All-Purpose Cluster." All-Purpose clusters are designed for interactive use and data exploration and can lead to resource contention with other jobs if used for production jobs. Option E fails for the same reason as option C and also has "Retries: None," which makes it prone to permanent failures.

In summary, a new job cluster, unlimited retries, and one concurrent run provides the best balance between fault tolerance and manageable costs for production Structured Streaming jobs on Databricks. The cluster is spun up only when needed, minimizing idle time costs, and retries ensure resilience.

For further research, consult Databricks documentation on:

Databricks Jobs: https://docs.databricks.com/workflows/jobs/index.html
Structured Streaming: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
Cluster Management: https://docs.databricks.com/clusters/index.html

Question: 4 **Deep Dumps**

The data engineering team has configured a Databricks SQL query and alert to monitor the values in a Delta Lake table. The recent_sensor_recordings table contains an identifying sensor_id alongside the timestamp and temperature for the most recent 5 minutes of recordings.
The below query is used to create the alert:

```
SELECT MEAN(temperature), MAX(temperature), MIN(temperature)
FROM recent_sensor_recordings
GROUP BY sensor_id
```

The query is set to refresh each minute and always completes in less than 10 seconds. The alert is set to trigger when mean (temperature) > 120. Notifications are triggered to be sent at most every 1 minute.
If this alert raises notifications for 3 consecutive minutes and then stops, which statement must be true?

A. The total average temperature across all sensors exceeded 120 on three consecutive executions of the query

B. The recent_sensor_recordings table was unresponsive for three consecutive runs of the query

C. The source query failed to update properly for three consecutive minutes and then restarted

D. The maximum temperature recording for at least one sensor exceeded 120 on three consecutive executions of the query

E. The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query

Answer: E

Explanation:

The average temperature recordings for at least one sensor exceeded 120 on three consecutive executions of the query.

Question: 5 **Deep Dumps**

A junior developer complains that the code in their notebook isn't producing the correct results in the development environment. A shared screenshot reveals that while they're using a notebook versioned with Databricks Repos, they're using a personal branch that contains old logic. The desired branch named dev-2.3.9 is not available from the branch selection dropdown.
Which approach will allow this developer to review the current logic for this notebook?

A.Use Repos to make a pull request use the Databricks REST API to update the current branch to dev-2.3.9

B.Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.

C.Use Repos to checkout the dev-2.3.9 branch and auto-resolve conflicts with the current branch

D.Merge all changes back to the main branch in the remote Git repository and clone the repo again

E.Use Repos to merge the current branch and the dev-2.3.9 branch, then make a pull request to sync with the remote repository

Answer: B

Explanation:

The correct approach is **B. Use Repos to pull changes from the remote Git repository and select the dev-2.3.9 branch.**

Here's a detailed justification:

The problem stems from the developer working with an outdated version of the code in their personal branch within Databricks Repos. The desired dev-2.3.9 branch, containing the correct logic, is not locally available. To rectify this, the developer needs to update their local repository with the latest changes from the remote repository where dev-2.3.9 resides.

Option B directly addresses this by using Repos' functionality to "pull" (fetch and merge) changes from the remote Git repository. This action downloads the latest commits and branches, including dev-2.3.9, to the developer's local repository. Once the pull is complete, the developer can then select (checkout) the dev-2.3.9 branch within Databricks Repos. This will switch the notebook to the version present in that branch, allowing the developer to review and use the correct logic.

Let's analyze why the other options are not suitable:

1. **A:** Making a pull request is intended for contributing changes, not for simply acquiring them. Also using the Databricks REST API to update the current branch feels overly complex for a simple git update.
2. **C:** Checking out the branch and auto-resolving conflicts could lead to unintended changes if the current branch has modifications the developer wants to discard or keep separate.
3. **D:** Merging to the main branch and recloning is a drastic measure and disrupts the established Git workflow. It's unnecessary to involve main for a localized branch update.
4. **E:** Merging the current branch with dev-2.3.9 before pulling could introduce unwanted conflicts if the developer's local branch contains errors.

Therefore, pulling changes from the remote repository and then selecting the correct branch is the most efficient and safest way for the junior developer to access the current logic.

Here are some links to further research Databricks Repos and Git operations:

1. **Databricks Repos Documentation:** https://docs.databricks.com/repos/index.html
2. **Git Basics - Getting a Git Repository:** https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository
3. **Git Branching - Basic Branching and Merging:** https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

Question: 6 **Deep Dumps**

The security team is exploring whether or not the Databricks secrets module can be leveraged for connecting to an external database.
After testing the code with all Python variables being defined with strings, they upload the password to the secrets module and configure the correct permissions for the currently active user. They then modify their code to the following (leaving all other variables unchanged).

```
password = dbutils.secrets.get(scope="db_creds", key="jdbc_password")

print(password)

df = (spark
  .read
  .format("jdbc")
  .option("url", connection)
  .option("dbtable", tablename)
  .option("user", username)
  .option("password", password)
  )
```

Which statement describes what will happen when the above code is executed?

A.The connection to the external table will fail; the string "REDACTED" will be printed.

B.An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the encoded password will be saved to DBFS.

C.An interactive input box will appear in the notebook; if the right password is provided, the connection will succeed and the password will be printed in plain text.

D.The connection to the external table will succeed; the string value of password will be printed in plain text.

E.The connection to the external table will succeed; the string "REDACTED" will be printed.

Answer: E

Explanation:

The connection to the external table will succeed; the string "REDACTED" will be printed.

https://learn.microsoft.com/en-us/azure/databricks/security/secrets/redaction

Question: 7 **Deep Dumps**

The data science team has created and logged a production model using MLflow. The following code correctly imports and applies the production model to output the predictions as a new DataFrame named preds with the schema "customer_id LONG, predictions DOUBLE, date DATE".

```
from pyspark.sql.functions import current_date

model = mlflow.pyfunc.spark_udf(spark, model_uri="models:/churn/prod")
df = spark.table("customers")
columns = ["account_age", "time_since_last_seen", "app_rating"]
preds = (df.select(
  "customer_id",
  model(*columns).alias("predictions"),
  current_date().alias("date")
  )
```

The data science team would like predictions saved to a Delta Lake table with the ability to compare all predictions across time. Churn predictions will be made at most once per day.
Which code block accomplishes this task while minimizing potential compute costs?

A.preds.write.mode("append").saveAsTable("churn_preds")

B.preds.write.format("delta").save("/preds/churn_preds")

```
(preds.writeStream
  .outputMode("overwrite")
  .option("checkpointPath", "/_checkpoints/churn_preds")
  .start("/preds/churn_preds")
)
```

C.

```
(preds.write
  .format("delta")
  .mode("overwrite")
  .saveAsTable("churn_preds")
)
```

D.

```
(preds.writeStream
  .outputMode("append")
  .option("checkpointPath", "/_checkpoints/churn_preds")
  .table("churn_preds")
)
```

E.

Answer: A

Explanation:

You need:- Batch operation since it is at most once a day- Append, since you need to keep track of past predictionsA is the correct answer. You don't need to specify "format" when you use saveAsTable.

Question: 8 **Deep Dumps**

An upstream source writes Parquet data as hourly batches to directories named with the current date. A nightly batch job runs the following code to ingest all data from the previous day as indicated by the date variable:

```
(spark.read
  .format("parquet")
  .load(f"/mnt/raw_orders/{date}")
  .dropDuplicates(["customer_id", "order_id"])
  .write
  .mode("append")
  .saveAsTable("orders")
)
```

Assume that the fields customer_id and order_id serve as a composite key to uniquely identify each order.
If the upstream system is known to occasionally produce duplicate entries for a single order hours apart, which statement is correct?

A.Each write to the orders table will only contain unique records, and only those records without duplicates in the target table will be written.

B.Each write to the orders table will only contain unique records, but newly written records may have duplicates

already present in the target table.

C.Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, these records will be overwritten.

D.Each write to the orders table will only contain unique records; if existing records with the same key are present in the target table, the operation will fail.

E.Each write to the orders table will run deduplication over the union of new and existing records, ensuring no duplicate records are present.

Answer: B

Explanation:

Each write to the orders table will only contain unique records, but newly written records may have duplicates already present in the target table.

Question: 9 **Deep Dumps**

A junior member of the data engineering team is exploring the language interoperability of Databricks notebooks. The intended outcome of the below code is to register a view of all sales that occurred in countries on the continent of Africa that appear in the geo_lookup table.
Before executing the code, running SHOW TABLES on the current database indicates the database contains only two tables: geo_lookup and sales.

```
Cmd 1
%python
countries_af = [x[0] for x in
spark.table("geo_lookup").filter("continent='AF'").select("country").collect()]

Cmd 2
%sql
CREATE VIEW sales_af AS
  SELECT *
  FROM sales
  WHERE city IN countries_af
  AND CONTINENT = "AF"
```

Which statement correctly describes the outcome of executing these command cells in order in an interactive notebook?

A.Both commands will succeed. Executing show tables will show that countries_af and sales_af have been registered as views.

B.Cmd 1 will succeed. Cmd 2 will search all accessible databases for a table or view named countries_af: if this entity exists, Cmd 2 will succeed.

C.Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable representing a PySpark DataFrame.

D.Both commands will fail. No new variables, tables, or views will be created.

E.Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

Answer: E

Explanation:

Cmd 1 will succeed and Cmd 2 will fail. countries_af will be a Python variable containing a list of strings.

Cmd 1 is a PySpark command that collects the list of countries from the 'geo_lookup' table where the continent is Africa ('AF'). This command will execute successfully, resulting in countries_af being a list of

country names (strings) in Python's local memory.

Cmd 2 is an SQL command intended to create a view named 'sales_af' from the 'sales' table, filtered by the cities in the countries_af list. However, this will fail because the countries_af variable exists in the Python environment and is not recognized in the SQL context. SQL does not have access to Python variables directly; they are two separate execution contexts within a Databricks notebook. There is no table or view named countries_af that SQL can reference; it is merely a Python list variable.

The other options are incorrect because they either assume cross-contextual operation between Python and SQL within a Databricks notebook (which is not possible in the way described in the commands), or they do not correctly interpret the outcome of running the commands.

Question: 10 **Deep Dumps**

A Delta table of weather records is partitioned by date and has the below schema: date DATE, device_id INT, temp FLOAT, latitude FLOAT, longitude FLOAT
To find all the records from within the Arctic Circle, you execute a query with the below filter: latitude > 66.3
Which statement describes how the Delta engine identifies which files to load?

A.All records are cached to an operational database and then the filter is applied
B.The Parquet file footers are scanned for min and max statistics for the latitude column
C.All records are cached to attached storage and then the filter is applied
D.The Delta log is scanned for min and max statistics for the latitude column
E.The Hive metastore is scanned for min and max statistics for the latitude column

Answer: D

Explanation:

The correct answer is D: The Delta log is scanned for min and max statistics for the latitude column.

Delta Lake enhances data lake reliability and performance by introducing a transaction log that meticulously records every change made to the data. This log contains metadata, including min/max statistics for each column within each data file. This enables Delta Lake to intelligently skip irrelevant files during query execution, a technique known as data skipping. When you execute a query with the filter latitude > 66.3, the Delta engine uses the transaction log to evaluate the min/max latitude values stored for each data file. If the minimum latitude value for a particular file exceeds 66.3, that file is guaranteed to contain relevant records and is included in the read set. Conversely, if the maximum latitude value for a file is less than or equal to 66.3, the file is skipped entirely, significantly reducing I/O operations and accelerating query performance. The Delta log eliminates the need to scan Parquet file footers directly (option B) or rely on external metastores like Hive for min/max statistics (option E). Options A and C are incorrect because they involve caching all records, which defeats the purpose of efficient data skipping. Delta Lake prioritizes reading only the data that satisfies the query condition.

For further research, consult these resources:

Delta Lake Documentation on Data Skipping: https://docs.databricks.com/delta/data-skipping.html
Delta Lake Under the Hood: https://databricks.com/blog/2019/08/22/diving-into-delta-lake-unpacking-the-transaction-log.html

Question: 11 **Deep Dumps**

The data engineering team has configured a job to process customer requests to be forgotten (have their data

deleted). All user data that needs to be deleted is stored in Delta Lake tables using default table settings.
The team has decided to process all deletions from the previous week as a batch job at 1am each Sunday. The total duration of this job is less than one hour. Every Monday at 3am, a batch job executes a series of VACUUM commands on all Delta Lake tables throughout the organization.
The compliance officer has recently learned about Delta Lake's time travel functionality. They are concerned that this might allow continued access to deleted data.
Assuming all delete logic is correctly implemented, which statement correctly addresses this concern?

A.Because the VACUUM command permanently deletes all files containing deleted records, deleted records may be accessible with time travel for around 24 hours.

B.Because the default data retention threshold is 24 hours, data files containing deleted records will be retained until the VACUUM job is run the following day.

C.Because Delta Lake time travel provides full access to the entire history of a table, deleted records can always be recreated by users with full admin privileges.

D.Because Delta Lake's delete statements have ACID guarantees, deleted records will be permanently purged from all storage systems as soon as a delete job completes.

E.Because the default data retention threshold is 7 days, data files containing deleted records will be retained until the VACUUM job is run 8 days later.

Answer: E

Explanation:

The correct answer is E because it accurately reflects Delta Lake's default data retention behavior and how it interacts with the VACUUM command in relation to data deletion and time travel.

Here's the detailed justification:

Delta Lake's time travel feature allows users to query older versions of a table. This is achieved by retaining older data files. By default, Delta Lake retains historical data for 7 days. This means data files containing records deleted in the last week will still be present in the underlying storage (e.g., Azure Blob Storage or AWS S3).

The VACUUM command is used to remove files from the Delta Lake table's underlying storage that are no longer referenced in the current table version and are older than the retention period. Without VACUUM, Delta Lake tables can accumulate a significant number of files over time, impacting query performance and storage costs.

The compliance officer's concern is valid. Even though records are deleted from the Delta Lake table, the underlying data files containing those deleted records persist for the retention period. Because the deletion job runs on Sunday at 1 am and the VACUUM job runs on Monday at 3 am the following week, this is 8 days later. Therefore, the files with deleted records will stick around for the retention period of 7 days, plus a little more time until the VACUUM command executes to remove those files.

Let's analyze why other options are incorrect:

A: While VACUUM does permanently delete files, it only does so after the retention period. The retention is not 24 hours by default.
B: The default retention is not 24 hours.
C: While admins can access history, the point is to minimize access to deleted data after the retention period, which VACUUM facilitates. This statement suggests data can always be recreated, which contradicts the purpose of data deletion and the VACUUM command.
D: Delta Lake's delete operations are ACID compliant, ensuring data consistency. However, ACID compliance doesn't mean immediate and permanent deletion from the storage system. The data still persists within the older versions for a defined period (retention period).

Therefore, option E accurately describes how Delta Lake's default retention policy affects the accessibility of deleted data via time travel and how the VACUUM command, run after the retention period, eventually

removes those files.

Authoritative Links:

Delta Lake Time Travel: https://docs.databricks.com/delta/versioning.html
Delta Lake VACUUM: https://docs.databricks.com/delta/vacuum.html

Question: 12 **Deep Dumps**

A junior data engineer has configured a workload that posts the following JSON to the Databricks REST API endpoint 2.0/jobs/create.

```
{
  "name": "Ingest new data",
  "existing_cluster_id": "6015-954420-peace720",
  "notebook_task": {
    "notebook_path": "/Prod/ingest.py"
  }
}
```

Assuming that all configurations and referenced resources are available, which statement describes the result of executing this workload three times?

A.Three new jobs named "Ingest new data" will be defined in the workspace, and they will each run once daily.

B.The logic defined in the referenced notebook will be executed three times on new clusters with the configurations of the provided cluster ID.

C.Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.

D.One new job named "Ingest new data" will be defined in the workspace, but it will not be executed.

E.The logic defined in the referenced notebook will be executed three times on the referenced existing all purpose cluster.

Answer: C

Explanation:

Three new jobs named "Ingest new data" will be defined in the workspace, but no jobs will be executed.

Question: 13 **Deep Dumps**

An upstream system is emitting change data capture (CDC) logs that are being written to a cloud object storage directory. Each record in the log indicates the change type (insert, update, or delete) and the values for each field after the change. The source table has a primary key identified by the field pk_id.
For auditing purposes, the data governance team wishes to maintain a full record of all values that have ever been valid in the source system. For analytical purposes, only the most recent value for each record needs to be recorded. The Databricks job to ingest these records occurs once per hour, but each individual record may have changed multiple times over the course of an hour.
Which solution meets these requirements?

A.Create a separate history table for each pk_id resolve the current state of the table by running a union all filtering the history tables for the most recent state.

B.Use MERGE INTO to insert, update, or delete the most recent entry for each pk_id into a bronze table, then propagate all changes throughout the system.

C.Iterate through an ordered set of changes to the table, applying each in turn; rely on Delta Lake's versioning ability to create an audit log.

D.Use Delta Lake's change data feed to automatically process CDC data from an external system, propagating all changes to all dependent tables in the Lakehouse.

E.Ingest all log information into a bronze table; use MERGE INTO to insert, update, or delete the most recent entry for each pk_id into a silver table to recreate the current table state.

Answer: D

Explanation:

The correct answer is **E** (Ingest all log information into a bronze table; use MERGE INTO to insert, update, or delete the most recent entry for each pk_id into a silver table to recreate the current table state.) because it provides a structured and scalable approach to handle CDC data while fulfilling both audit and analytical requirements.

Here's a detailed justification:

1. **Bronze Table for Raw Data (Audit):** The initial ingestion into a bronze table ensures that all CDC logs are captured exactly as they arrive from the upstream system. This fulfills the audit requirement of maintaining a full record of all values that have ever been valid. The bronze table acts as the immutable source of truth, preserving all changes for historical analysis and compliance.
2. **Silver Table for Current State (Analytical):** The silver table is designed to hold the most recent value for each record. This caters to the analytical needs of the data governance team.
3. **MERGE INTO for Upserts:** The use of MERGE INTO operation on the silver table efficiently handles inserts, updates, and deletes based on the pk_id. Within each hourly batch, the MERGE INTO operation effectively overwrites older values for a given pk_id with the latest one found in the CDC logs, ensuring the silver table always reflects the most current state.
4. **Scalability and Performance:** This approach is scalable because Delta Lake and MERGE INTO are optimized for large datasets. The incremental processing within each hourly batch allows for efficient updates without requiring a full table scan.

Why other options are less suitable:

A: Creating a separate history table for each pk_id is unscalable and makes querying the current state extremely complex due to the need for a UNION ALL operation and complex filtering.
B: Propagating all changes from a bronze table directly is not suitable as the system requires two different views of the data - the full history, and the current state.
C: While Delta Lake's versioning provides an audit log, iterating through an ordered set of changes and applying them manually is less efficient and harder to manage than using MERGE INTO.
D: Delta Lake Change Data Feed (CDF) is powerful, but doesn't directly address the requirement to maintain a full historical audit log in addition to the most recent state without additional processing steps. CDF captures changes from a Delta table, not changes to a target Delta table based on external CDC.

Relevant Cloud Computing Concepts:

Data Lakehouse: The overall architecture leverages the Data Lakehouse paradigm, combining the benefits of data lakes (scalability, flexibility) and data warehouses (structure, governance). Bronze and silver tables represent different layers in the Lakehouse architecture.
Change Data Capture (CDC): The solution is specifically designed to handle CDC data, which is a common pattern for integrating data from operational systems.
Upsert Operation: MERGE INTO is an upsert operation that efficiently combines inserts and updates into a single operation, crucial for handling CDC data.
Delta Lake: Delta Lake provides ACID properties and versioning to the data lake, enabling reliable and scalable data processing.

Authoritative Links:

Delta Lake MERGE INTO: https://docs.databricks.com/en/delta/delta-update.html#use-merge-into
Delta Lake Change Data Feed: https://docs.databricks.com/en/delta/delta-change-data-feed.html
Data Lakehouse: https://databricks.com/glossary/data-lakehouse

Question: 14 **Deep Dumps**

An hourly batch job is configured to ingest data files from a cloud object storage container where each batch represent all records produced by the source system in a given hour. The batch job to process these records into the Lakehouse is sufficiently delayed to ensure no late-arriving data is missed. The user_id field represents a unique key for the data, which has the following schema: user_id BIGINT, username STRING, user_utc STRING, user_region STRING, last_login BIGINT, auto_pay BOOLEAN, last_updated BIGINT
New records are all ingested into a table named account_history which maintains a full record of all data in the same schema as the source. The next table in the system is named account_current and is implemented as a Type 1 table representing the most recent value for each unique user_id.
Assuming there are millions of user accounts and tens of thousands of records processed hourly, which implementation can be used to efficiently update the described account_current table as part of each hourly batch job?

A.Use Auto Loader to subscribe to new files in the account_history directory; configure a Structured Streaming trigger once job to batch update newly detected files into the account_current table.

B.Overwrite the account_current table with each batch using the results of a query against the account_history table grouping by user_id and filtering for the max value of last_updated.

C.Filter records in account_history using the last_updated field and the most recent hour processed, as well as the max last_iogin by user_id write a merge statement to update or insert the most recent value for each user_id.

D.Use Delta Lake version history to get the difference between the latest version of account_history and one version prior, then write these records to account_current.

E.Filter records in account_history using the last_updated field and the most recent hour processed, making sure to deduplicate on username; write a merge statement to update or insert the most recent value for each username.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most efficient and appropriate solution for updating the account_current table, along with explanations and relevant links.

Justification for Option C: Filter and Merge

Option C offers the most efficient method for maintaining a Type 1 slowly changing dimension (SCD) table like account_current given the provided scenario. Here's why:

1. **Incremental Processing:** It avoids processing the entire account_history table every hour. By filtering account_history based on the last_updated field and the most recent hour, you significantly reduce the data volume needing processing. This is crucial for performance, especially with millions of user accounts and tens of thousands of hourly records.
2. **Efficient Deduplication:** By identifying the maximum last_login for each user_id within the hourly batch, the solution ensures that only the most recent update for each user is considered. This prevents older or redundant records from overwriting the most current data in account_current.
3. **MERGE Statement for UPSERT:** The MERGE statement is designed for efficiently updating existing records and inserting new records in a single operation (UPSERT - Update or Insert). This is more performant than separate update and insert operations. Delta Lake's optimized MERGE implementation further enhances this efficiency. The MERGE statement is preferred as it efficiently

combines the update and insert operations based on matching keys.
4. **Key-Based Update:** Using user_id as the key in the MERGE statement ensures that updates are applied to the correct user's record in the account_current table.

Why other options are less ideal:

1. **A (Auto Loader with Streaming):** While Auto Loader is great for ingestion, using Structured Streaming with a trigger once batch job adds unnecessary complexity and overhead. Batch processing is already in place, so using streaming is not the most direct approach. Also, it does not naturally solve the de-duplication needed.
2. **B (Overwrite):** Overwriting the entire account_current table every hour is highly inefficient, particularly with millions of users. It requires reprocessing all data, even if only a small subset has changed. This also loses the time travel capabilities that Delta lake brings to the table.
3. **D (Delta Lake Version History):** While Delta Lake version history is useful, calculating the difference between versions requires reading and comparing potentially large datasets. Using last_updated and MERGE is more direct and targeted. Also, this approach does not account for users who may have had multiple updates in a single batch.
4. **E (Deduplicate on username):** Deduplicating on username is incorrect, as the problem statement uses user_id as the unique key for accounts. The username might change.

Relevant Cloud Computing Concepts:

1. **Slowly Changing Dimensions (SCDs):** account_current is a Type 1 SCD, meaning it always reflects the most current data.
2. **UPSERT (Update or Insert):** The MERGE statement performs UPSERT operations efficiently.
3. **Delta Lake:** Delta Lake enhances data reliability and performance through ACID transactions, schema enforcement, and time travel.
4. **Incremental Data Processing:** Processing only the changed or new data, as opposed to the full dataset, is a core principle for efficient data pipelines.

Authoritative Links:

1. **Delta Lake MERGE:** https://docs.databricks.com/delta/delta-update.html#upsert-into-a-delta-table-using-merge
2. **Slowly Changing Dimensions:** https://en.wikipedia.org/wiki/Slowly_changing_dimension

In summary, option C provides the most efficient and scalable solution by leveraging incremental processing, deduplication, and the MERGE statement to update the account_current table based on hourly batches of data.

Question: 15 **Deep Dumps**

A table in the Lakehouse named customer_churn_params is used in churn prediction by the machine learning team. The table contains information about customers derived from a number of upstream sources. Currently, the data engineering team populates this table nightly by overwriting the table with the current valid values derived from upstream data sources.
The churn prediction model used by the ML team is fairly stable in production. The team is only interested in making predictions on records that have changed in the past 24 hours.
Which approach would simplify the identification of these changed records?

A.Apply the churn model to all rows in the customer_churn_params table, but implement logic to perform an upsert into the predictions table that ignores rows where predictions have not changed.

B.Convert the batch job to a Structured Streaming job using the complete output mode; configure a Structured Streaming job to read from the customer_churn_params table and incrementally predict against the churn model.

C.Calculate the difference between the previous model predictions and the current customer_churn_params on a key identifying unique customers before making new predictions; only make predictions on those customers not in the previous predictions.

D.Modify the overwrite logic to include a field populated by calling spark.sql.functions.current_timestamp() as data are being written; use this field to identify records written on a particular date.

E.Replace the current overwrite logic with a merge statement to modify only those records that have changed; write logic to make predictions on the changed records identified by the change data feed.

Answer: E

Explanation:

The correct answer is **E** because it offers the most efficient and reliable way to identify and process changed records for churn prediction. Let's break down why:

Change Data Capture (CDC) with MERGE: Option E leverages CDC via a MERGE statement. This is a core data warehousing concept specifically designed to track and propagate changes in data sources.

Efficiency: By using MERGE, you only update records that have actually changed. This avoids processing the entire table every night, significantly reducing computational resources and time.

Change Data Feed (CDF): Change Data Feed (CDF) provides a record of every change made to a Delta table, allowing you to efficiently identify which rows have been inserted, updated, or deleted. By writing logic to make predictions only on the changed records identified by the CDF, you dramatically reduce the scope of the prediction task.

Simplified Identification: MERGE inherently identifies changed records. You can easily extract these changes and flag them for prediction. No complex comparison logic is needed.

Why other options are less suitable:

A: Applying the model to all rows is wasteful and defeats the purpose of optimizing for changed records. Upserts can be complex and add unnecessary overhead.
B: Structured Streaming with complete output mode is not efficient for this use case. Complete mode requires reprocessing the entire table on each update, which negates the benefits of identifying changed records.
C: Calculating the difference between predictions introduces complexities in maintaining and comparing prediction histories. It's not a standard or efficient practice.
D: While current_timestamp() provides a timestamp, it doesn't inherently capture changes. Records might have the same timestamp but unchanged data. This is a workaround and not a robust CDC solution. It doesn't track specific columns that changed.

In Summary: Option E effectively implements CDC using MERGE and the change data feed to identify, isolate, and process changed records. It's the most performant, maintainable, and scalable solution for this scenario. Using CDC aligns with best practices for data warehousing and enables incremental processing, thus providing a simpler and more resource-efficient solution.

Authoritative Links:

Delta Lake Change Data Feed: https://docs.databricks.com/delta/delta-change-data-feed.html
Delta Lake MERGE: https://docs.databricks.com/delta/delta-update.html#upsert-into-a-delta-table-using-merge

Question: 16 **Deep Dumps**

A table is registered with the following code:

```
CREATE TABLE recent_orders AS (
  SELECT a.user_id, a.email, b.order_id, b.order_date
  FROM
    (SELECT user_id, email
    FROM users) a
    INNER JOIN
    (SELECT user_id, order_id, order_date
    FROM orders
    WHERE order_date >= (current_date() - 7)) b
    ON a.user_id = b.user_id
)
```

Both users and orders are Delta Lake tables. Which statement describes the results of querying recent_orders?

A.All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query finishes.

B.All logic will execute when the table is defined and store the result of joining tables to the DBFS; this stored data will be returned when the table is queried.

C.Results will be computed and cached when the table is defined; these cached results will incrementally update as new records are inserted into source tables.

D.All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

E.The versions of each source table will be stored in the table transaction log; query results will be saved to DBFS with each query.

Answer: D

Explanation:

All logic will execute at query time and return the result of joining the valid versions of the source tables at the time the query began.

Question: 17 **Deep Dumps**

A production workload incrementally applies updates from an external Change Data Capture feed to a Delta Lake table as an always-on Structured Stream job. When data was initially migrated for this table, OPTIMIZE was executed and most data files were resized to 1 GB. Auto Optimize and Auto Compaction were both turned on for the streaming production job. Recent review of data files shows that most data files are under 64 MB, although each partition in the table contains at least 1 GB of data and the total table size is over 10 TB.
Which of the following likely explains these smaller file sizes?

A.Databricks has autotuned to a smaller target file size to reduce duration of MERGE operations

B.Z-order indices calculated on the table are preventing file compaction

C.Bloom filter indices calculated on the table are preventing file compaction

D.Databricks has autotuned to a smaller target file size based on the overall size of data in the table

E.Databricks has autotuned to a smaller target file size based on the amount of data in each partition

Answer: A

Explanation:

The most likely explanation for the observed smaller file sizes (under 64MB) in the Delta Lake table, despite initial optimization to 1GB files and Auto Optimize/Compaction being enabled, is that Databricks has autotuned to a smaller target file size to optimize MERGE operations.

Here's a detailed justification:

MERGE Operation Impact: MERGE operations are used to update Delta tables based on Change Data Capture (CDC) feeds. These operations can be I/O intensive and slow, especially with large files.
Auto Tuning for MERGE: Delta Lake's auto-tuning mechanism monitors the performance of MERGE operations. When it detects that MERGE operations are taking longer than expected, especially related to the size of the base files being updated, it can automatically adjust the target file size downwards.
Smaller File Size Benefits: Reducing the target file size results in more frequent but smaller compactions. This approach makes MERGE operations more efficient because they need to rewrite smaller amounts of data during updates.
Auto Optimize and Compaction: Auto Optimize generally takes the overall size of the delta table into account to optimize the number of files. Auto compaction consolidates small files. But since the data is constantly being updated through the stream job, autotuning likely took precedence.
Partition Size: While each partition contains at least 1 GB of data, the smaller file sizes suggest the optimization is happening at a finer granularity to specifically accelerate the MERGE operations.
Other Options:

Z-order and Bloom Filters: Z-order clustering and bloom filters can affect query performance, but they don't directly prevent file compaction.
Table Size as Primary Factor: Autotuning generally considers the table's overall size, but it primarily responds to the performance characteristics of ongoing operations, such as MERGE. Therefore, the size of the table itself isn't necessarily the primary trigger.

Therefore, the continuous CDC feed and resulting MERGE operations likely caused Databricks to automatically reduce the target file size to improve update performance, even though Auto Optimize and Compaction were initially configured.

Relevant Links:

Delta Lake Auto Optimize
Delta Lake MERGE

Question: 18 **Deep Dumps**

Which statement regarding stream-static joins and static Delta tables is correct?

A.Each microbatch of a stream-static join will use the most recent version of the static Delta table as of each microbatch.

B.Each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization.

C.The checkpoint directory will be used to track state information for the unique keys present in the join.

D.Stream-static joins cannot use static Delta tables because of consistency issues.

E.The checkpoint directory will be used to track updates to the static Delta table.

Answer: B

Explanation:

The correct statement regarding stream-static joins and static Delta tables is that each microbatch of a stream-static join will use the most recent version of the static Delta table as of the job's initialization. This

behavior is crucial for ensuring consistency during the streaming process. When a streaming job starts, it reads the static Delta table. During each micro-batch, the streaming data is joined against the same snapshot of the static table that was available at the job's start.

This design choice avoids inconsistencies that could arise from using different versions of the static table across different micro-batches. It also improves performance, as the static table is only read once at the beginning, rather than for every microbatch. This contrasts with stream-stream joins, which need to handle updates on both sides. The checkpoint directory primarily stores state information related to the streaming data and aggregation/transformation logic, not updates to the static Delta table. While Delta Lake ensures ACID properties for the static table, the stream-static join mechanism uses a single snapshot for the entire streaming job's duration. Therefore, the statement that stream-static joins cannot use static Delta tables due to consistency issues is incorrect; they are designed to work together by utilizing a consistent snapshot of the static data.

For more information, please refer to the official Databricks documentation on structured streaming and Delta Lake. Specifically, consider these pages:

Structured Streaming with Delta Lake: This is a general starting point.
Delta Lake and Structured Streaming: Blog posts discussing how Delta Lake can be used in streaming pipelines.
Structured Streaming Programming Guide: Covers the fundamentals of structured streaming.

These resources will provide deeper insights into stream-static joins and how they interact with Delta Lake.

Question: 19 **Deep Dumps**

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Events are recorded once per minute per device.
Streaming DataFrame df has the following schema:
"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"
Code block:

```
df.withWatermark("event_time", "10 minutes")
    .groupBy(
        _____,
        "device_id"
    )
    .agg(
        avg("temp").alias("avg_temp"),
        avg("humidity").alias("avg_humidity")
    )
    .writeStream
    .format("delta")
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

A.to_interval("event_time", "5 minutes").alias("time")
B.window("event_time", "5 minutes").alias("time")
C."event_time"
D.window("event_time", "10 minutes").alias("time")
E.lag("event_time", "10 minutes").alias("time")

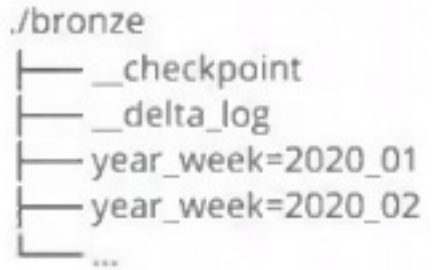
Answer: B

Explanation:

window("event_time", "5 minutes").alias("time")

Question: 20 **Deep Dumps**

A data architect has designed a system in which two Structured Streaming jobs will concurrently write to a single bronze Delta table. Each job is subscribing to a different topic from an Apache Kafka source, but they will write data with the same schema. To keep the directory structure simple, a data engineer has decided to nest a checkpoint directory to be shared by both streams.
The proposed directory structure is displayed below:



Which statement describes whether this checkpoint directory structure is valid for the given scenario and why?

A.No; Delta Lake manages streaming checkpoints in the transaction log.

B.Yes; both of the streams can share a single checkpoint directory.

C.No; only one stream can write to a Delta Lake table.

D.Yes; Delta Lake supports infinite concurrent writers.

E.No; each of the streams needs to have its own checkpoint directory.

Answer: E

Explanation:

No; each of the streams needs to have its own checkpoint directory.

Reference:

https://docs.databricks.com/en/optimizations/isolation-level.html#:~:text=If%20a%20streaming%20query%20using%20the%20same%20checkpoint%20location%

20is%20started%20multiple%20times%20concurrently%20and%20tries%20to%20write%20to%20the%20Delta%

20table%20at%20the%20same%20time.%20You%20should%20never%20have%20two%20streaming%

20queries%20use%20the%20same%20checkpoint%20location%20and%20run%20at%20the%20same%20time.

Question: 21 **Deep Dumps**

A Structured Streaming job deployed to production has been experiencing delays during peak hours of the day. At present, during normal execution, each microbatch of data is processed in less than 3 seconds. During peak hours of the day, execution time for each microbatch becomes very inconsistent, sometimes exceeding 30 seconds. The streaming write is currently configured with a trigger interval of 10 seconds.
Holding all other variables constant and assuming records need to be processed in less than 10 seconds, which adjustment will meet the requirement?

A.Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish.

B.Increase the trigger interval to 30 seconds; setting the trigger interval near the maximum execution time observed for each batch is always best practice to ensure no records are dropped.

C.The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism.

D.Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch.

E.Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.

Answer: E

Explanation:

Here's a breakdown of why option E is the most appropriate solution and why the others are less suitable for addressing the streaming job delays:

The core problem is that during peak hours, micro-batch processing times increase significantly, exceeding the desired 10-second limit. The goal is to ensure records are processed within this time frame to prevent backlog and maintain near real-time processing.

Why Option E is Correct: Decrease the trigger interval to 5 seconds; triggering batches more frequently may prevent records from backing up and large batches from causing spill.

Preventing Backlog: By reducing the trigger interval (e.g., from 10 seconds to 5 seconds), the system ingests and processes data more frequently. This helps to avoid large accumulations of data waiting to be processed, especially during peak periods when the input rate increases. Smaller batches reduce the strain on executors.
Reducing Spill: When batches become excessively large (due to infrequent triggering and higher input rates), they might exceed available memory, leading to "spilling" data to disk. Disk I/O is significantly slower than memory access, drastically increasing processing time. Smaller batches (with more frequent triggers) reduce the likelihood of spilling.
Improved Resource Utilization: Frequent triggering allows executors to switch to new batches as soon as they're ready, thus preventing idle time.
Example: Imagine a conveyor belt carrying packages. If the belt only moves every 10 seconds, packages pile up if the arrival rate is high. If the belt moves every 5 seconds, the pile-up is less severe.

Why Other Options are Incorrect:

Option A: Decrease the trigger interval to 5 seconds; triggering batches more frequently allows idle executors to begin processing the next batch while longer running tasks from previous batches finish. This answer contains a partial truth. While decreasing the trigger interval is a viable option, the reasoning for this is slightly misleading as the executors working on the longer running tasks of the previous batch cannot begin processing the new batch as they are still busy. Option E provides a much more thorough description of the benefits.
Option B: Increase the trigger interval to 30 seconds; setting the trigger interval near the maximum execution time observed for each batch is always best practice to ensure no records are dropped. Increasing the trigger interval is completely counterproductive. It exacerbates the problem by creating even larger batches, which will take even longer to process and increase the risk of spilling. Also, increasing the trigger to 30 seconds while the requirement is processing records in less than 10 seconds immediately makes this an unviable option.
Option C: The trigger interval cannot be modified without modifying the checkpoint directory; to maintain the current stream state, increase the number of shuffle partitions to maximize parallelism. This is factually incorrect. The trigger interval can be changed dynamically without modifying the checkpoint directory. This is because the trigger interval affects the frequency, not the state. Although increasing shuffle partitions can

improve parallelism, it does not directly address the issue of peak hour delays stemming from the stream configuration. Additionally, increasing shuffle partitions may not necessarily lead to a performance improvement if the data isn't properly partitioned.
Option D: Use the trigger once option and configure a Databricks job to execute the query every 10 seconds; this ensures all backlogged records are processed with each batch. trigger once is designed for one-time data processing rather than continuous streaming. While using Databricks jobs scheduled every 10 seconds sounds similar, it isn't the same as a structured stream. Trigger once will process all available data and then stop, requiring re-scheduling. Also, scheduling it with Databricks jobs loses the fault-tolerance and state management capabilities inherent in Structured Streaming. This also doesn't solve the root cause of long processing times.

Supporting Concepts:

Structured Streaming: A scalable and fault-tolerant stream processing engine built on Apache Spark. https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
Micro-Batching: Structured Streaming processes data in small batches (micro-batches) at specified intervals.
Trigger Interval: Determines how frequently new micro-batches are created and processed.
Backpressure: Mechanism for handling situations where the rate of incoming data exceeds the processing capacity. While not explicitly mentioned in the answer options, this is closely related.
Resource Utilization: Efficiently using available compute resources (CPU, memory) to maximize throughput and minimize latency.

In summary, decreasing the trigger interval is the most direct and effective way to address the micro-batch processing delays during peak hours, preventing backlogs, reducing the risk of spills, and optimizing resource utilization.

Question: 22 **Deep Dumps**

Which statement describes Delta Lake Auto Compaction?

A.An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 1 GB.

B.Before a Jobs cluster terminates, OPTIMIZE is executed on all tables modified during the most recent job.

C.Optimized writes use logical partitions instead of directory partitions; because partition boundaries are only represented in metadata, fewer small files are written.

D.Data is queued in a messaging bus instead of committing data directly to memory; all data is committed from the messaging bus in one batch once the job is complete.

E.An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 128 MB.

Answer: E

Explanation:

Delta Lake Auto Compaction is a crucial feature that optimizes the storage layout of Delta tables. The correct statement is **E. An asynchronous job runs after the write completes to detect if files could be further compacted; if yes, an OPTIMIZE job is executed toward a default of 128 MB.**

Here's why this is the correct explanation:

Asynchronous Process: Auto Compaction runs independently and after write operations (inserts, updates, deletes). This means it doesn't block or delay the write transactions themselves.
File Size Monitoring: The auto compaction process monitors the size of the files in the Delta table. It identifies files that are smaller than a threshold size (small files).

OPTIMIZE Command Trigger: If the auto compaction detects an opportunity to compact files, it essentially runs the OPTIMIZE command on the table in the background. The OPTIMIZE command consolidates small files into larger ones.
Default Target Size: The key detail that distinguishes option E from option A is the target file size after compaction. Delta Lake auto compaction aims to produce files around 128 MB in size by default. This is much smaller than the 1GB in Option A which is likely to be for whole-table optimization jobs.

The goal of auto compaction is to reduce the number of small files in the Delta table. A large number of small files can negatively impact query performance. Reading numerous small files requires more metadata operations and can lead to increased I/O overhead. By consolidating these files into larger ones, auto compaction improves query speed and efficiency.

Options A, B, C, and D are incorrect for the following reasons:

Option A: The default target file size of 1 GB is incorrect. The correct default is 128 MB.
Option B: This describes a completely different and non-existent feature. OPTIMIZE is not automatically executed when a Jobs cluster terminates.
Option C: Optimized Writes and partition handling are different features that don't involve Auto Compaction. Optimized writes do help minimize small file creation from the start, rather than afterward.
Option D: Using a messaging bus like Kafka for direct data committing is not relevant to the function of Delta Lake Auto Compaction.

Authoritative Links:

Databricks Documentation on Auto Compaction: https://docs.databricks.com/delta/optimizations/auto-optimize.html
Databricks Blog on Delta Lake Optimizations: (search Databricks blog for "Delta Lake optimization techniques") - this contains relevant blog posts to explain the concepts.

Question: 23 **Deep Dumps**

Which statement characterizes the general programming model used by Spark Structured Streaming?

A.Structured Streaming leverages the parallel processing of GPUs to achieve highly parallel data throughput.

B.Structured Streaming is implemented as a messaging bus and is derived from Apache Kafka.

C.Structured Streaming uses specialized hardware and I/O streams to achieve sub-second latency for data transfer.

D.Structured Streaming models new data arriving in a data stream as new rows appended to an unbounded table.

E.Structured Streaming relies on a distributed network of nodes that hold incremental state values for cached stages.

Answer: D

Explanation:

The correct answer is D because it accurately describes the fundamental programming model of Spark Structured Streaming. Spark Structured Streaming treats a live data stream as a continuously appending table. As new data arrives, it's considered as new rows being added to this unbounded table. This abstraction allows users to apply familiar batch-oriented SQL queries and DataFrame operations to streaming data, making it easier to process.

Option A is incorrect because while Spark can utilize GPUs for specific computations, Structured Streaming doesn't inherently depend on them for general parallel processing. Spark primarily uses its distributed

execution engine for parallel data processing, leveraging clusters of CPUs. Option B is incorrect as Structured Streaming is built on top of the Spark SQL engine, not derived from Apache Kafka. While Kafka is a common data source for streaming applications using Spark, it's not the foundation of Structured Streaming's implementation. Kafka acts as a messaging system, while Structured Streaming provides the processing framework. Option C is inaccurate as Structured Streaming focuses on fault-tolerant and scalable processing using Spark's distributed architecture, rather than relying on specialized hardware or I/O streams for low latency. Latency is addressed through micro-batching or continuous processing mode within Spark's fault-tolerant framework. Option E describes a general concept of caching and distributed state management in Spark, but it doesn't specifically define the programming model of Structured Streaming. Spark does maintain state information for certain operations (like windowing or aggregations), but the core programming model revolves around the unbounded table abstraction.

In essence, Structured Streaming aims to unify batch and streaming processing by providing a declarative API to define data pipelines, abstracting away many of the complexities of stream processing and allowing developers to focus on the data transformation logic. This unified approach is a key characteristic of modern data engineering practices.

Further research can be found at:

Apache Spark Structured Streaming Documentation: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html
Databricks Documentation on Structured Streaming: https://docs.databricks.com/structured-streaming/index.html

Question: 24 **Deep Dumps**

Which configuration parameter directly affects the size of a spark-partition upon ingestion of data into Spark?

A.spark.sql.files.maxPartitionBytes
B.spark.sql.autoBroadcastJoinThreshold
C.spark.sql.files.openCostInBytes
D.spark.sql.adaptive.coalescePartitions.minPartitionNum
E.spark.sql.adaptive.advisoryPartitionSizeInBytes

Answer: A

Explanation:

Here's a detailed justification for why option A, spark.sql.files.maxPartitionBytes, is the configuration parameter that directly affects the size of a Spark partition during data ingestion:

When Spark reads data from files, it divides the data into partitions for parallel processing. The size of these partitions significantly impacts performance. If partitions are too small, the overhead of managing them overwhelms the processing time. If they are too large, it can lead to memory issues on executors and reduce parallelism.

spark.sql.files.maxPartitionBytes directly controls the maximum number of bytes Spark attempts to pack into a single partition when reading data from file-based sources like Parquet, Avro, or CSV. Spark aims to create partitions as close to this size as possible without splitting individual files. Increasing this value generally reduces the number of partitions created and increases the size of each partition, and vice versa.

Option B, spark.sql.autoBroadcastJoinThreshold, affects whether Spark uses a broadcast join strategy, which impacts how data is distributed for joins, not the initial partition size during data ingestion.

Option C, spark.sql.files.openCostInBytes, influences how Spark estimates the cost of opening a file. This value plays a role in determining the optimal number of files to read per task, affecting partitioning but less directly than maxPartitionBytes. It helps Spark avoid opening too many small files in one task but doesn't explicitly set a target partition size.

Option D, spark.sql.adaptive.coalescePartitions.minPartitionNum, governs the minimum number of partitions after adaptive query execution (AQE) coalesces partitions. It's relevant post-ingestion and during query optimization, not the initial partitioning.

Option E, spark.sql.adaptive.advisoryPartitionSizeInBytes, acts as a suggestion or hint to AQE during the query optimization phase. It doesn't directly determine the initial partition size at the time of data ingestion. AQE might use it to guide its repartitioning decisions, but the initial partitioning is handled separately.

Therefore, spark.sql.files.maxPartitionBytes is the most direct and influential parameter for controlling Spark partition size during data ingestion from file-based sources.

Authoritative Links:

Apache Spark Configuration: https://spark.apache.org/docs/latest/configuration.html (Search for the specific properties to find the documentation.)
Databricks Documentation on Spark Configuration: https://docs.databricks.com/optimizations/tune-spark-jobs.html (This provides broader guidance on Spark tuning within the Databricks environment.)

Question: 25 **Deep Dumps**

A Spark job is taking longer than expected. Using the Spark UI, a data engineer notes that the Min, Median, and Max Durations for tasks in a particular stage show the minimum and median time to complete a task as roughly the same, but the max duration for a task to be roughly 100 times as long as the minimum.
Which situation is causing increased duration of the overall job?

A.Task queueing resulting from improper thread pool assignment.
B.Spill resulting from attached volume storage being too small.
C.Network latency due to some cluster nodes being in different regions from the source data
D.Skew caused by more data being assigned to a subset of spark-partitions.
E.Credential validation errors while pulling data from an external system.

Answer: D

Explanation:

The correct answer is **D. Skew caused by more data being assigned to a subset of spark-partitions.**

Here's why: The Spark UI showing a vastly higher maximum task duration compared to the minimum and median durations within a stage strongly indicates **data skew**. Data skew occurs when some partitions in a Spark DataFrame/RDD contain significantly more data than others. This imbalance leads to some tasks (processing the larger partitions) taking much longer to complete than others. The "min" and "median" durations representing the tasks that process smaller, "normal" sized partitions will be similar. However, the "max" duration represents the task processing the disproportionately large partition, resulting in the observed 100x difference.

Let's eliminate the other options:

A. Task queueing resulting from improper thread pool assignment: While thread pool misconfiguration can cause delays, it generally affects all tasks fairly evenly. You wouldn't typically see a massive difference in the duration of individual tasks, but rather a delay before tasks start. Queueing would affect all tasks across

executors rather than certain partitions disproportionately.

B. Spill resulting from attached volume storage being too small: Spill can cause performance degradation, but it would likely lead to more uniformly slowed tasks rather than extreme variations. Also, spill is limited by the executor memory, not attached volume storage. The spark.local.dir configuration can direct the local dir to spill to attached volume storage. While the total task time would be affected, the difference between min/median/max wouldn't be as drastic as 100x if only spill was the problem. Also, it affects all tasks, not just a few.

C. Network latency due to some cluster nodes being in different regions from the source data: Network latency might increase overall job duration, but would again likely affect most tasks somewhat evenly. While certain partitions might have slight network latency, it is unlikely to cause specific task to take 100x longer than others.

E. Credential validation errors while pulling data from an external system: These errors would likely lead to task failures or potentially indefinite delays, but not consistently long task durations. It would be more of an all-or-nothing situation.

Data skew directly addresses the specific symptom of widely varying task durations. The partitions with very large data amounts cause individual tasks to take exponentially more time to process. Mitigating data skew involves techniques such as salting, bucketing, or repartitioning the data to distribute it more evenly across partitions before the stage in question.

Supporting Links:

Apache Spark Performance Tuning: Data Skew: https://spark.apache.org/docs/latest/tuning.html#data-skew - This link provides detailed information on how to deal with data skew.
Databricks - Handling Skewed Data: https://www.databricks.com/blog/2015/07/22/handling-skewed-data-in-spark.html

Question: 26 **Deep Dumps**

Each configuration below is identical to the extent that each cluster has 400 GB total of RAM, 160 total cores and only one Executor per VM.
Given a job with at least one wide transformation, which of the following cluster configurations will result in maximum performance?

A.• Total VMs; 1
• 400 GB per Executor
• 160 Cores / Executor

B.• Total VMs: 8
• 50 GB per Executor
• 20 Cores / Executor

C.• Total VMs: 16
• 25 GB per Executor
• 10 Cores/Executor

D.• Total VMs: 4
• 100 GB per Executor
• 40 Cores/Executor

E.• Total VMs:2
• 200 GB per Executor
• 80 Cores / Executor

Answer: A

Explanation:

The optimal cluster configuration for maximum performance in a Spark job with wide transformations, given the constraints, leans towards minimizing network overhead and maximizing resource locality. While all options have the same total RAM and cores, the distribution across VMs impacts performance differently.

Option A, with a single large VM, consolidates all data and processing within one node. Wide transformations, which require shuffling data across the cluster, benefit from this locality because all data transfer occurs internally within the VM's memory. This minimizes network latency, a significant bottleneck in distributed computing.

Options B, C, D, and E involve multiple VMs. Distributing the data across multiple machines introduces network overhead during the shuffle phase of wide transformations. The more VMs, the more data must traverse the network, increasing latency and decreasing performance. While having more executors can offer increased parallelism for certain operations, the penalty incurred from network shuffles outweighs the benefit in this scenario where network transfer is the major bottleneck, due to a wide transformation.

Furthermore, having only one executor per VM simplifies resource allocation and avoids potential resource contention within a single VM. This can improve predictability and stability of the job.

Therefore, option A, utilizing a single VM with all resources, offers the highest performance by eliminating network overhead during wide transformations, even though it might seemingly offer less parallelism at first glance. The benefits of local data access outweigh the reduced parallelism, which would have otherwise required data to be passed through the network.

For more information on Spark performance tuning, see:

Spark Performance Tuning
Databricks Spark Optimization (While specific to Databricks, the concepts apply broadly).

Question: 27 **Deep Dumps**

A junior data engineer on your team has implemented the following code block.

```
MERGE INTO events
USING new_events
ON events.event_id = new_events.event_id
WHEN NOT MATCHED
  INSERT *
```

The view new_events contains a batch of records with the same schema as the events Delta table. The event_id field serves as a unique key for this table.
When this query is executed, what will happen with new records that have the same event_id as an existing record?

A.They are merged.
B.They are ignored.
C.They are updated.
D.They are inserted.
E.They are deleted.

Answer: B

Explanation:

Correct answer is B:They are ignored.

Reference:

https://docs.databricks.com/en/sql/language-manual/delta-merge-into.html#:~:text=If%20none%20of%20the%20WHEN%20MATCHED%20conditions%20evaluate%20to%20true%

Question: 28 **Deep Dumps**

A junior data engineer seeks to leverage Delta Lake's Change Data Feed functionality to create a Type 1 table representing all of the values that have ever been valid for all rows in a bronze table created with the property delta.enableChangeDataFeed = true. They plan to execute the following code as a daily job:

```
from pyspark.sql.functions import col

(spark.read.format("delta")
  .option("readChangeFeed", "true")
  .option("startingVersion", 0)
  .table("bronze")
  .filter(col("_change_type").isin(["update_postimage", "insert"]))
  .write
  .mode("append")
  .table("bronze_history_type1")
)
```

Which statement describes the execution and results of running the above query multiple times?

A.Each time the job is executed, newly updated records will be merged into the target table, overwriting previous values with the same primary keys.

B.Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.

C.Each time the job is executed, the target table will be overwritten using the entire history of inserted or updated records, giving the desired result.

D.Each time the job is executed, the differences between the original and current versions are calculated; this may result in duplicate entries for some records.

E.Each time the job is executed, only those records that have been inserted or updated since the last execution will be appended to the target table, giving the desired result.

Answer: B

Explanation:

Each time the job is executed, the entire available history of inserted or updated records will be appended to the target table, resulting in many duplicate entries.

Delta Lake's Change Data Feed (CDF) functionality allows tracking and capturing changes (inserts, updates, and deletes) in a table. When leveraging this functionality, it's important to ensure that only the new or changed records are processed to avoid duplicates.

The provided job appears to capture all changes from the bronze table and append them to the target table. However, if the job is executed daily without filtering out records that have already been processed, the entire history of changes will be appended each time, leading to multiple duplicate entries.

Question: 29 **Deep Dumps**

A new data engineer notices that a critical field was omitted from an application that writes its Kafka source to Delta Lake. This happened even though the critical field was in the Kafka source. That field was further missing from data written to dependent, long-term storage. The retention threshold on the Kafka service is seven days. The pipeline has been in production for three months.
Which describes how Delta Lake can help to avoid data loss of this nature in the future?

A.The Delta log and Structured Streaming checkpoints record the full history of the Kafka producer.

B.Delta Lake schema evolution can retroactively calculate the correct value for newly added fields, as long as the data was in the original source.

C.Delta Lake automatically checks that all fields present in the source data are included in the ingestion layer.

D.Data can never be permanently dropped or deleted from Delta Lake, so data loss is not possible under any circumstance.

E.Ingesting all raw data and metadata from Kafka to a bronze Delta table creates a permanent, replayable history of the data state.

Answer: E

Explanation:

The correct answer is E because ingesting raw Kafka data and its metadata into a bronze Delta table provides a complete and immutable record of the original data source. This approach ensures that even if downstream pipelines or transformations inadvertently omit a field, the original data is still available.

Here's a detailed justification:

Bronze Layer as Source of Truth: The bronze layer in a medallion architecture acts as the source of truth, containing the raw, unprocessed data. Storing Kafka data in this layer ensures that no data is lost during initial ingestion.
Immutability and Replayability: Delta Lake's append-only nature guarantees that data is not overwritten or deleted. This immutability, combined with Delta Lake's versioning capabilities, allows for replaying the Kafka stream from any point in time, even after the retention period on Kafka has expired, as the data is persisted in long-term storage.
Historical Data Access: By retaining the raw data, it becomes possible to retroactively correct errors or omissions in downstream pipelines. If the critical field was present in the original Kafka stream, it can be retrieved from the bronze table and propagated to silver and gold tables.
Metadata Preservation: Saving metadata alongside the data is crucial. This metadata can include timestamps, offsets, and other Kafka-specific information, which can be invaluable for debugging and auditing purposes.
Avoids Irreversible Data Loss: If the data engineering team realizes after three months that a critical field was missing, they can replay the Kafka stream from the bronze table to generate a new version of the silver and gold tables containing the critical field.

Why other options are incorrect:

A: Delta Lake and Structured Streaming checkpoints do not contain the full history of the Kafka producer's data; they are operational metadata for stream processing.
B: Delta Lake schema evolution is useful for adapting to changes in the data schema, but it cannot magically recreate missing data if it wasn't ingested in the first place. Schema evolution updates the table schema to accommodate new fields, but it doesn't populate historical data with values that were never present.
C: Delta Lake does not enforce that all source fields must be present. It can be configured to enforce schema, but it's not automatic.
D: While Delta Lake protects against accidental deletion and provides time travel, it doesn't prevent data loss caused by incorrect data ingestion or transformation logic. It doesn't guarantee data is never dropped under any circumstance. For instance, an improperly configured transformation could still lead to data exclusion.

Authoritative Links:

Delta Lake Bronze to Gold: https://www.databricks.com/blog/2019/08/15/diving-into-delta-lake-unpacking-the-transaction-log.html
Medallion Architecture: https://www.databricks.com/glossary/medallion-architecture
Structured Streaming + Delta Lake: https://spark.apache.org/docs/latest/structured-streaming-delta-lake-integration.html

Question: 30 **Deep Dumps**

A nightly job ingests data into a Delta Lake table using the following code:

```
from pyspark.sql.functions import current_timestamp, input_file_name, col
from pyspark.sql.column import Column

def ingest_daily_batch(time_col: Column, year:int, month:int, day:int):
    (spark.read
        .format("parquet")
        .load(f"/mnt/daily_batch/{year}/{month}/{day}")
        .select("*",
                time_col.alias("ingest_time"),
                inpute_file_name().alias("source_file")
               )
        .write
        .mode("append")
        .saveAsTable("bronze")
    )
```

The next step in the pipeline requires a function that returns an object that can be used to manipulate new records that have not yet been processed to the next table in the pipeline.
Which code snippet completes this function definition?
def new_records():

A.return spark.readStream.table("bronze")

B.return spark.readStream.load("bronze")

```
return (spark.read
    .table("bronze")
    .filter(col("ingest_time") == current_timestamp())
)
```

C.

D.return spark.read.option("readChangeFeed", "true").table ("bronze")

E.

```
return (spark.read
    .table("bronze")
    .filter(col("source_file") == f"/mnt/daily_batch/{year}/{month}/{day}")
)
```

Answer: D

Explanation:

return spark.read.option("readChangeFeed", "true").table ("bronze")

Question: 31 **Deep Dumps**

A junior data engineer is working to implement logic for a Lakehouse table named silver_device_recordings. The source data contains 100 unique fields in a highly nested JSON structure.
The silver_device_recordings table will be used downstream to power several production monitoring dashboards and a production model. At present, 45 of the 100 fields are being used in at least one of these applications.
The data engineer is trying to determine the best approach for dealing with schema declaration given the highly-nested structure of the data and the numerous fields.
Which of the following accurately presents information about Delta Lake and Databricks that may impact their decision-making process?

A.The Tungsten encoding used by Databricks is optimized for storing string data; newly-added native support for querying JSON strings means that string types are always most efficient.

B.Because Delta Lake uses Parquet for data storage, data types can be easily evolved by just modifying file footer information in place.

C.Human labor in writing code is the largest cost associated with data engineering workloads; as such, automating table declaration logic should be a priority in all migration workloads.

D.Because Databricks will infer schema using types that allow all observed data to be processed, setting types manually provides greater assurance of data quality enforcement.

E.Schema inference and evolution on Databricks ensure that inferred types will always accurately match the data types used by downstream systems.

Answer: D

Explanation:

The correct answer is D. Here's why:

Option D highlights a crucial aspect of data quality and control in a data lakehouse environment. While Databricks offers schema inference, it prioritizes processing data and automatically infers the schema based on the first few records. This inferred schema might use more generic data types to accommodate all observed values. For instance, if a column sometimes contains integers and sometimes strings, the inferred type might be string to avoid data loss. However, this can lead to unexpected behavior and inconsistencies downstream, potentially breaking production dashboards or models that expect specific data types. Manually defining the schema enforces strict data type constraints, ensuring that only valid data conforming to the expected schema is ingested. This promotes data quality, prevents errors, and ensures consistency for downstream applications. This is especially important when dealing with production systems where reliability and accuracy are paramount. By explicitly defining the schema, the data engineer gains greater control over the data quality and guarantees that the data aligns with the expectations of the production monitoring dashboards and the production model.

Let's analyze why the other options are incorrect:

A: While Databricks uses Tungsten for optimized data processing, relying solely on string types for all data, even with JSON support, is generally inefficient. String storage and parsing can consume more resources compared to using appropriate native data types (integer, boolean, etc.) for each field, especially when performing aggregations or calculations.

B: Delta Lake does use Parquet, but schema evolution in Delta Lake involves updating the Delta transaction log and potentially rewriting data files, not just modifying file footers. It is a more involved process for data consistency.

C: While automation is important, it should not come at the expense of careful design and data quality. In critical systems, manual intervention and verification may be necessary to ensure the accuracy and reliability of the data. The need to consider the specific data characteristics and the downstream application requirements outweighs the cost of manual code.

E: Schema inference in Databricks does not guarantee perfect type matching with downstream systems. Downstream systems might have different expectations or limitations, necessitating explicit schema definition and data type conversions.

Therefore, manually setting the schema provides more control over data quality and ensures compatibility with downstream applications, which is particularly vital for production systems.

Supporting links:

Delta Lake Schema Evolution: https://docs.databricks.com/delta/delta-schema.html
Databricks Data Types: https://spark.apache.org/docs/latest/sql-ref.html#data-types

Question: 32 **Deep Dumps**

The data engineering team maintains the following code:

```
accountDF = spark.table("accounts")
orderDF = spark.table("orders")
itemDF = spark.table("items")

orderWithItemDF = (orderDF.join(
    itemDF,
    orderDF.itemID == itemDF.itemID)
  .select(
    orderDF.accountID,
    orderDF.itemID,

    itemDF.itemName))

finalDF = (accountDF.join(
    orderWithItemDF,
    accountDF.accountID == orderWithItemDF.accountID)
  .select(
    orderWithItemDF["*"],

    accountDF.city))

(finalDF.write
  .mode("overwrite")
  .table("enriched_itemized_orders_by_account"))
```

Assuming that this code produces logically correct results and the data in the source tables has been de-duplicated and validated, which statement describes what will occur when this code is executed?

A.A batch job will update the enriched_itemized_orders_by_account table, replacing only those rows that have different values than the current version of the table, using accountID as the primary key.

B.The enriched_itemized_orders_by_account table will be overwritten using the current valid version of data in

each of the three tables referenced in the join logic.

C.An incremental job will leverage information in the state store to identify unjoined rows in the source tables and write these rows to the enriched_iteinized_orders_by_account table.

D.An incremental job will detect if new rows have been written to any of the source tables; if new rows are detected, all results will be recalculated and used to overwrite the enriched_itemized_orders_by_account table.

E.No computation will occur until enriched_itemized_orders_by_account is queried; upon query materialization, results will be calculated using the current valid version of data in each of the three tables referenced in the join logic.

Answer: B

Explanation:

The enriched_itemized_orders_by_account table will be overwritten using the current valid version of data in each of the three tables referenced in the join logic.

Assuming that the code is logically correct and the source data is already de-duplicated and validated, the code likely includes a join operation to combine data from multiple tables (itemized_orders, account_info, and perhaps another source) to create or refresh the enriched_itemized_orders_by_account table.

Question: 33 **Deep Dumps**

The data engineering team is migrating an enterprise system with thousands of tables and views into the Lakehouse. They plan to implement the target architecture using a series of bronze, silver, and gold tables. Bronze tables will almost exclusively be used by production data engineering workloads, while silver tables will be used to support both data engineering and machine learning workloads. Gold tables will largely serve business intelligence and reporting purposes. While personal identifying information (PII) exists in all tiers of data, pseudonymization and anonymization rules are in place for all data at the silver and gold levels.
The organization is interested in reducing security concerns while maximizing the ability to collaborate across diverse teams.
Which statement exemplifies best practices for implementing this system?

A.Isolating tables in separate databases based on data quality tiers allows for easy permissions management through database ACLs and allows physical separation of default storage locations for managed tables.

B.Because databases on Databricks are merely a logical construct, choices around database organization do not impact security or discoverability in the Lakehouse.

C.Storing all production tables in a single database provides a unified view of all data assets available throughout the Lakehouse, simplifying discoverability by granting all users view privileges on this database.

D.Working in the default Databricks database provides the greatest security when working with managed tables, as these will be created in the DBFS root.

E.Because all tables must live in the same storage containers used for the database they're created in, organizations should be prepared to create between dozens and thousands of databases depending on their data isolation requirements.

Answer: A

Explanation:

The best practice for implementing the Lakehouse architecture with bronze, silver, and gold tables, considering security, collaboration, and data quality tiers, is isolating tables in separate databases based on data quality tiers. This approach aligns with the principle of least privilege, improving security by restricting access to specific data sets based on user roles and responsibilities.

Databases in Databricks (and more broadly in cloud data platforms) are not merely logical constructs; they provide a crucial level of namespace isolation and access control. By creating separate databases for bronze,

silver, and gold layers, administrators can grant granular permissions through database Access Control Lists (ACLs). This prevents unauthorized users from accessing sensitive raw data in the bronze layer while still granting access to pseudonymized or anonymized data in silver and gold layers.

Furthermore, separate databases allow for physical separation of storage locations for managed tables. This isolation can be crucial for compliance and regulatory requirements, allowing you to enforce different retention policies and encryption settings for each data tier. It's also beneficial for performance optimization; you can tailor storage settings based on the specific workload accessing each tier.

Option B is incorrect because databases significantly impact security and discoverability through ACLs and metadata organization. Option C, storing all production tables in a single database, undermines security by making it difficult to restrict access to sensitive raw data. Option D is incorrect; working in the default database is not secure and lacks proper organization. Finally, option E is impractical and inefficient. The number of databases should be based on logical data groupings and security requirements, not simply mirroring the number of tables. Database proliferation increases management overhead.

Therefore, option A offers the best balance of security, discoverability, and manageability by leveraging databases for logical and physical isolation of data tiers with varying sensitivity levels and use cases.

Relevant links:

Databricks documentation on Data Governance and Security: https://www.databricks.com/glossary/data-governance
Azure Data Lake Storage Gen2 Access Control Model: https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-access-control-model

Question: 34 **Deep Dumps**

The data architect has mandated that all tables in the Lakehouse should be configured as external Delta Lake tables.
Which approach will ensure that this requirement is met?

A.Whenever a database is being created, make sure that the LOCATION keyword is used
B.When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.
C.Whenever a table is being created, make sure that the LOCATION keyword is used.
D.When tables are created, make sure that the EXTERNAL keyword is used in the CREATE TABLE statement.
E.When the workspace is being configured, make sure that external cloud object storage has been mounted.

Answer: C

Explanation:

The correct answer is C: "Whenever a table is being created, make sure that the LOCATION keyword is used." This ensures that the Delta Lake table is external, meaning the data files are stored in a location outside the Delta Lake metastore's managed location.

Here's a detailed justification:

External vs. Managed Tables: Delta Lake supports both managed and external tables. Managed tables' data files are stored in a default location managed by the Delta Lake metastore (e.g., the Hive metastore or Unity Catalog). External tables, on the other hand, have their data files stored in a user-specified location.

The LOCATION Keyword: The LOCATION keyword in the CREATE TABLE statement explicitly specifies the storage location for the table's data files. This forces the creation of an external table. Without the LOCATION keyword, Delta Lake defaults to creating a managed table and storing the data within its own managed

directory structure.

Why Other Options are Incorrect:

A: While specifying a location during database creation can influence where managed tables within that database are stored, it doesn't inherently guarantee all tables are external. Tables can still be created without a location within that database, reverting to default managed behavior.
B: Configuring an external data warehouse and using Databricks for ELT doesn't directly enforce that all Delta Lake tables are external. The table creation process still needs to explicitly define the storage location.
D: There is no EXTERNAL keyword in standard Delta Lake CREATE TABLE syntax that automatically makes the table external. The LOCATION keyword achieves this.
E: Mounting external cloud object storage only provides access to the storage; it doesn't automatically create external tables. Tables still need to be created with a LOCATION referring to the mounted storage to be external.

Meeting the Data Architect's Mandate: By consistently using the LOCATION keyword in every CREATE TABLE statement, you ensure that all tables created are external Delta Lake tables, thus adhering to the data architect's requirement. This allows for greater control over data storage locations and independent management of data files.

For further research, refer to the following resources:

CREATE TABLE [USING]: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-table-using.html - Pay attention to the LOCATION keyword.
Delta Lake Table Management: (Search for "managed vs. external tables" on the Databricks documentation) https://docs.databricks.com/delta/index.html

Question: 35 **Deep Dumps**

To reduce storage and compute costs, the data engineering team has been tasked with curating a series of aggregate tables leveraged by business intelligence dashboards, customer-facing applications, production machine learning models, and ad hoc analytical queries.
The data engineering team has been made aware of new requirements from a customer-facing application, which is the only downstream workload they manage entirely. As a result, an aggregate table used by numerous teams across the organization will need to have a number of fields renamed, and additional fields will also be added.
Which of the solutions addresses the situation while minimally interrupting other teams in the organization without increasing the number of tables that need to be managed?

A.Send all users notice that the schema for the table will be changing; include in the communication the logic necessary to revert the new table schema to match historic queries.

B.Configure a new table with all the requisite fields and new names and use this as the source for the customer-facing application; create a view that maintains the original data schema and table name by aliasing select fields from the new table.

C.Create a new table with the required schema and new fields and use Delta Lake's deep clone functionality to sync up changes committed to one table to the corresponding table.

D.Replace the current table definition with a logical view defined with the query logic currently writing the aggregate table; create a new table to power the customer-facing application.

E.Add a table comment warning all users that the table schema and field names will be changing on a given date; overwrite the table in place to the specifications of the customer-facing application.

Answer: B

Explanation:

The correct solution is B because it addresses the requirements with minimal disruption and without unnecessary duplication of data. Here's a detailed justification:

Minimizing disruption: Option B maintains the original schema for existing users through a view. A view is a virtual table based on a query, allowing users to continue querying the original table name and schema without modification. This avoids breaking existing dashboards, applications, and queries relying on the original table structure.

Meeting new requirements: The new table with the required fields and names serves the customer-facing application's specific needs. This ensures the application has the data it needs in the required format, preventing modification of the original table that would impact other users.

Avoiding data duplication: Using a view avoids creating a completely separate copy of the data. The view simply aliases and selects from the new table, which is the single source of truth for the underlying data. This minimizes storage costs and ensures data consistency.

Other options' drawbacks:

Option A: While notifying users is important, it shifts the burden of schema adaptation to all existing users. This is disruptive and requires them to modify their queries, leading to increased operational overhead.

Option C: Deep cloning is more appropriate for creating backups or isolated environments, not for schema evolution. It doubles the storage required, which is contrary to the goal of cost reduction.

Option D: Replacing the table with a view may impact performance, especially if the underlying query is complex. Also, creating another table specifically for one application introduces unnecessary complexity.

Option E: Overwriting the table in place will break all downstream workloads using the original schema and is therefore highly disruptive. A table comment isn't sufficient mitigation.

In summary, option B leverages the flexibility of views to provide schema compatibility for existing users while adapting the data structure to meet the new customer-facing application's requirements, minimizing disruption and cost.

Here are some authoritative links for further research:

Databricks Views: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-view.html
Delta Lake Deep Clone: https://docs.databricks.com/delta/clone.html

Question: 36 **Deep Dumps**

A Delta Lake table representing metadata about content posts from users has the following schema: user_id LONG, post_text STRING, post_id STRING, longitude FLOAT, latitude FLOAT, post_time TIMESTAMP, date DATE
This table is partitioned by the date column. A query is run with the following filter: longitude < 20 & longitude > -20
Which statement describes how data will be filtered?

A.Statistics in the Delta Log will be used to identify partitions that might Include files in the filtered range.

B.No file skipping will occur because the optimizer does not know the relationship between the partition column and the longitude.

C.The Delta Engine will use row-level statistics in the transaction log to identify the flies that meet the filter criteria.

D.Statistics in the Delta Log will be used to identify data files that might include records in the filtered range.

E.The Delta Engine will scan the parquet file footers to identify each row that meets the filter criteria.

Answer: D

Explanation:

The correct answer is D: Statistics in the Delta Log will be used to identify data files that might include

records in the filtered range. This is because Delta Lake maintains metadata, including statistics, about each data file in the Delta Log. These statistics include min/max values for each column within the data file.

When a query with a filter on longitude (longitude < 20 & longitude > -20) is executed, Delta Lake's query optimizer leverages these statistics. It checks the longitude min/max values for each data file recorded in the Delta Log. If the specified range (-20 to 20) overlaps with the data file's longitude range, the data file might contain relevant records.

This process enables data skipping. Delta Lake only reads data files that potentially contain records satisfying the longitude filter condition, significantly reducing the amount of data scanned and improving query performance. The partition column, date, is not relevant in this scenario because the filter is on the longitude column, not the date column. Row-level statistics aren't used; Delta Lake relies on aggregate statistics per file for skipping. Parquet footers are not directly scanned for skipping; metadata in the Delta Log is used.

Option A is partially correct but focuses on partitions, not data files. Option B is incorrect because file skipping will occur based on longitude statistics. Option C refers to row-level statistics, which are generally not used for skipping. Option E describes inefficient behavior, contradicting Delta Lake's optimization capabilities.

Therefore, Delta Lake's capability to use the Delta Log statistics to identify potentially relevant data files is the key to efficient data filtering.

Relevant documentation:

Delta Lake Statistics and Data Skipping: https://docs.databricks.com/delta/data-skipping.html
Delta Lake Optimization: https://docs.databricks.com/delta/optimizations/index.html

Question: 37 **Deep Dumps**

A small company based in the United States has recently contracted a consulting firm in India to implement several new data engineering pipelines to power artificial intelligence applications. All the company's data is stored in regional cloud storage in the United States.
The workspace administrator at the company is uncertain about where the Databricks workspace used by the contractors should be deployed.
Assuming that all data governance considerations are accounted for, which statement accurately informs this decision?

A.Databricks runs HDFS on cloud volume storage; as such, cloud virtual machines must be deployed in the region where the data is stored.

B.Databricks workspaces do not rely on any regional infrastructure; as such, the decision should be made based upon what is most convenient for the workspace administrator.

C.Cross-region reads and writes can incur significant costs and latency; whenever possible, compute should be deployed in the same region the data is stored.

D.Databricks leverages user workstations as the driver during interactive development; as such, users should always use a workspace deployed in a region they are physically near.

E.Databricks notebooks send all executable code from the user’s browser to virtual machines over the open internet; whenever possible, choosing a workspace region near the end users is the most secure.

Answer: C

Explanation:

The optimal Databricks workspace location hinges on minimizing latency and costs associated with data access. Option C correctly identifies this.

Here's why:

Data Locality: Placing compute resources (Databricks clusters) closer to the data storage reduces the physical distance data needs to travel. This minimizes latency, the delay in data transfer, which is crucial for performance-sensitive AI pipelines.

Network Costs: Cloud providers typically charge for data transfer between regions (cross-region egress). Consolidating compute and storage within the same region avoids these egress charges, leading to significant cost savings, especially with large datasets.

Performance Impact: High latency can significantly slow down data processing tasks, negatively impacting the performance of data engineering pipelines and downstream AI applications. Deploying the workspace in the same region ensures faster data reads and writes, optimizing performance.

Options A, B, D, and E are incorrect:

Option A: Databricks does not run HDFS directly on cloud volume storage in the way suggested. It uses cloud object storage like AWS S3, Azure Blob Storage, or Google Cloud Storage directly.
Option B: Databricks workspace location definitely matters due to data locality and associated costs.
Option D: While user workstation proximity can be relevant for interactive notebook responsiveness, it's secondary to the data locality consideration for production data pipelines. The driver is on the cluster, not the user's machine.
Option E: Code is not sent from the user's browser to the compute cluster over the open internet. Databricks uses secure connections and internal network infrastructure within the cloud provider.

In conclusion, for efficient data pipelines, aligning the Databricks workspace location with the data storage region is paramount, primarily due to reduced latency and cost optimization.

Further Reading:

Azure Databricks Regions: https://azure.microsoft.com/en-us/explore/global-infrastructure/regions/ (Illustrates regional considerations in cloud deployments)
AWS Data Transfer Pricing: https://aws.amazon.com/ec2/pricing/on-demand/ (Example of cost implications related to data transfer)
Google Cloud Data Transfer Pricing: https://cloud.google.com/products/calculator/ (Example of cost implications related to data transfer)

Question: 38 **Deep Dumps**

The downstream consumers of a Delta Lake table have been complaining about data quality issues impacting performance in their applications. Specifically, they have complained that invalid latitude and longitude values in the activity_details table have been breaking their ability to use other geolocation processes.
A junior engineer has written the following code to add CHECK constraints to the Delta Lake table:

```
ALTER TABLE activity_details
ADD CONSTRAINT valid_coordinates
CHECK (
  latitude >= -90 AND
  latitude <= 90 AND
  longitude >= -180 AND
  longitude <= 180);
```

A senior engineer has confirmed the above logic is correct and the valid ranges for latitude and longitude are provided, but the code fails when executed.
Which statement explains the cause of this failure?

A.Because another team uses this table to support a frequently running application, two-phase locking is preventing the operation from committing.

B.The activity_details table already exists; CHECK constraints can only be added during initial table creation.

C.The activity_details table already contains records that violate the constraints; all existing data must pass CHECK constraints in order to add them to an existing table.

D.The activity_details table already contains records; CHECK constraints can only be added prior to inserting values into a table.

E.The current table schema does not contain the field valid_coordinates; schema evolution will need to be enabled before altering the table to add a constraint.

Answer: C

Explanation:

The correct option is C, with constraints, if added to an existing table the existing data in the table must be consistent with the constraint otherwise it fails.

https://docs.databricks.com/en/sql/language-manual/sql-ref-syntax-ddl-alter-table.html#add-constraint

Question: 39 **Deep Dumps**

Which of the following is true of Delta Lake and the Lakehouse?

A.Because Parquet compresses data row by row. strings will only be compressed when a character is repeated multiple times.

B.Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.

C.Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times.

D.Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table.

E.Z-order can only be applied to numeric values stored in Delta Lake tables.

Answer: B

Explanation:

Here's a detailed justification for why option B is the correct answer, along with supporting information and links.

Justification for Option B: Delta Lake automatically collects statistics on the first 32 columns of each table which are leveraged in data skipping based on query filters.

Delta Lake enhances data lake reliability and performance through several key features, and the automatic collection of statistics is a crucial aspect. Delta Lake automatically collects statistics on the first 32 columns of a Delta table when the table is created or updated. These statistics include min/max values, null counts, and other metadata for each column within each data file. This information is then used by the query optimizer for data skipping.

Data skipping significantly improves query performance. When a query has a filter condition, the query optimizer leverages the collected statistics to determine which data files do not contain relevant data based on the filter criteria. These irrelevant files are then skipped during query execution, resulting in a significant reduction in the amount of data read and processed. This reduces I/O operations, CPU usage, and overall query latency.

Why the other options are incorrect:

A. Because Parquet compresses data row by row. strings will only be compressed when a character is

repeated multiple times. Parquet uses columnar storage, not row-based storage. Columnar storage allows for better compression, especially for repeating values in the same column. String compression in Parquet is not limited to repeated characters within a row.
C. Views in the Lakehouse maintain a valid cache of the most recent versions of source tables at all times. Views are logical constructs defined by a query. They do not automatically maintain a cache of the most recent versions of source tables. Views are evaluated at query time.
D. Primary and foreign key constraints can be leveraged to ensure duplicate values are never entered into a dimension table. While constraints are being developed, Delta Lake, as of the current knowledge cutoff, does not fully enforce primary key and foreign key constraints in the same way that traditional relational databases do. You can define them, but they are not automatically enforced during write operations to prevent duplicates.
E. Z-order can only be applied to numeric values stored in Delta Lake tables. Z-ordering can be applied to string columns as well, not just numeric columns. Z-ordering is a technique to improve data locality and query performance by co-locating related data on disk based on multiple columns.

Supporting Links:

Databricks Delta Lake Documentation - Data Skipping: https://docs.databricks.com/optimizations/data-skipping.html
Databricks Delta Lake Documentation - Z-Ordering: https://docs.databricks.com/optimizations/z-order.html
Databricks Blog - Deep Dive into Delta Lake Uniqueness and Constraint Management: https://www.databricks.com/blog/2023/03/14/deep-dive-delta-lake-uniqueness-and-constraint-management.html

These resources provide in-depth explanations of Delta Lake's data skipping, Z-ordering, and constraint management capabilities. Remember to always refer to the latest Databricks documentation for the most up-to-date information.

Question: 40 **Deep Dumps**

The view updates represents an incremental batch of all newly ingested data to be inserted or updated in the customers table.
The following logic is used to process these records.

```
MERGE INTO customers
USING (
  SELECT updates.customer_id as merge_ey, updates.*
  FROM updates

  UNION ALL

  SELECT NULL as merge_key, updates.*
  FROM updates JOIN customers
  ON updates.customer_id = customers.customer_id
  WHERE customers.current = true AND updates.address <> customers.address
) staged_updates
ON customers.customer_id = mergeKey
WHEN MATCHED AND customers.current = true AND customers.address <> staged_updates.address THEN
  UPDATE SET current = false, end_date = staged_updates.effective_date
WHEN NOT MATCHED THEN
  INSERT(customer_id, address, current, effective_date, end_date)
  VALUES(staged_updates.customer_id, staged_updates.address, true, staged_updates.effective_date,
null)
```

Which statement describes this implementation?

A.The customers table is implemented as a Type 3 table; old values are maintained as a new column alongside the current value.

B.The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.

C.The customers table is implemented as a Type 0 table; all writes are append only with no changes to existing values.

D.The customers table is implemented as a Type 1 table; old values are overwritten by new values and no history is maintained.

E.The customers table is implemented as a Type 2 table; old values are overwritten and new customers are appended.

Answer: B

Explanation:

The customers table is implemented as a Type 2 table; old values are maintained but marked as no longer current and new values are inserted.

Question: 41 **Deep Dumps**

The DevOps team has configured a production workload as a collection of notebooks scheduled to run daily using the Jobs UI. A new data engineering hire is onboarding to the team and has requested access to one of these notebooks to review the production logic.
What are the maximum notebook permissions that can be granted to the user without allowing accidental changes to production code or data?

A.Can Manage

B.Can Edit

C.No permissions

D.Can Read

E.Can Run

Answer: D

Explanation:

The correct answer is D, "Can Read." Here's why:

The core objective is to grant access for reviewing production logic without risking unintended modifications to the code or the resulting data. The principle of least privilege dictates granting only the minimum necessary permissions to achieve the task.

A. **Can Manage:** "Manage" permissions provide the user with complete control over the notebook. They can edit, run, delete, change permissions, and essentially do anything with the notebook. This clearly violates the requirement to prevent accidental changes to production code.

B. **Can Edit:** "Edit" permissions, as the name suggests, allow the user to modify the notebook's content. This directly contradicts the requirement to avoid accidental code changes. Granting "Edit" access presents a high risk of inadvertently altering the production logic, which is unacceptable.

C. **No permissions:** This would prevent the new hire from reviewing the code, making it impossible for them to onboard or understand the production logic, therefore, failing to meet the user's requirements.

D. **Can Read:** "Read" permissions allow the user to view the notebook's contents, including the code and any markdown explanations. They cannot modify the notebook in any way. This is the ideal permission level for code review as it satisfies the requirement to review the production logic while preventing any accidental changes to production notebooks and data.

E. **Can Run:** "Run" permissions allows the user to execute the notebook. While they cannot change the code, running the production notebook could inadvertently trigger data processing or modifications within the environment, which can lead to unintended side effects in the production system. Since the requirement is only to review the code, running the notebook is not necessary and adds unnecessary risk.

Therefore, "Can Read" is the most restrictive permission level that still allows the user to accomplish the requested task of reviewing the notebook content. This aligns with the principle of least privilege and safeguards the production environment.

Relevant Documentation:

Databricks Notebook Permissions: While specific documentation directly detailing the impacts of each permission level on Jobs is limited, general Databricks ACL and Permissions documentation can give insight. Searching the Databricks documentation for "Access Control Lists" or "ACLs" and "Notebook Permissions" will provide related information.

Question: 42 **Deep Dumps**

A table named user_ltv is being used to create a view that will be used by data analysts on various teams. Users in the workspace are configured into groups, which are used for setting up data access using ACLs.
The user_ltv table has the following schema:
email STRING, age INT, ltv INT
The following view definition is executed:

```
CREATE VIEW email_ltv AS
SELECT
CASE WHEN
  is_member('marketing') THEN email
  ELSE 'REDACTED'
END AS email,
ltv
FROM user_ltv
```

An analyst who is not a member of the marketing group executes the following query:

SELECT * FROM email_ltv -
Which statement describes the results returned by this query?

A.Three columns will be returned, but one column will be named "REDACTED" and contain only null values.

B.Only the email and ltv columns will be returned; the email column will contain all null values.

C.The email and ltv columns will be returned with the values in user_ltv.

D.The email.age, and ltv columns will be returned with the values in user_ltv.

E.Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each row.

Answer: E

Explanation:

Only the email and ltv columns will be returned; the email column will contain the string "REDACTED" in each

row.

Question: 43 **Deep Dumps**

The data governance team has instituted a requirement that all tables containing Personal Identifiable Information (PH) must be clearly annotated. This includes adding column comments, table comments, and setting the custom table property "contains_pii" = true.
The following SQL DDL statement is executed to create a new table:

```
CREATE TABLE dev.pii_test
(id INT, name STRING COMMENT "PII")
COMMENT "Contains PII"
TBLPROPERTIES ('contains_pii' = True)
```

Which command allows manual confirmation that these three requirements have been met?

A.DESCRIBE EXTENDED dev.pii_test

B.DESCRIBE DETAIL dev.pii_test

C.SHOW TBLPROPERTIES dev.pii_test

D.DESCRIBE HISTORY dev.pii_test

E.SHOW TABLES dev

Answer: A

Explanation:

DESCRIBE EXTENDED dev.pii_test.

Question: 44 **Deep Dumps**

The data governance team is reviewing code used for deleting records for compliance with GDPR. They note the following logic is used to delete records from the Delta Lake table named users.

```
DELETE FROM users
WHERE user_id IN
  (SELECT user_id FROM delete_requests)
```

Assuming that user_id is a unique identifying key and that delete_requests contains all users that have requested deletion, which statement describes whether successfully executing the above logic guarantees that the records to be deleted are no longer accessible and why?

A.Yes; Delta Lake ACID guarantees provide assurance that the DELETE command succeeded fully and permanently purged these records.

B.No; the Delta cache may return records from previous versions of the table until the cluster is restarted.

C.Yes; the Delta cache immediately updates to reflect the latest data files recorded to disk.

D.No; the Delta Lake DELETE command only provides ACID guarantees when combined with the MERGE INTO command.

E.No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

Answer: E

Explanation:

No; files containing deleted records may still be accessible with time travel until a VACUUM command is used to remove invalidated data files.

Question: 45 **Deep Dumps**

An external object storage container has been mounted to the location /mnt/finance_eda_bucket.
The following logic was executed to create a database for the finance team:

```
CREATE DATABASE finance_eda_db
LOCATION '/mnt/finance_eda_bucket';
GRANT USAGE ON DATABASE finance_eda_db TO finance;
GRANT CREATE ON DATABASE finance_eda_db TO finance;
```

After the database was successfully created and permissions configured, a member of the finance team runs the following code:

```
CREATE TABLE finance_eda_db.tx_sales AS
SELECT *
FROM sales
WHERE state = "TX";
```

If all users on the finance team are members of the finance group, which statement describes how the tx_sales table will be created?

A.A logical table will persist the query plan to the Hive Metastore in the Databricks control plane.

B.An external table will be created in the storage container mounted to /mnt/finance_eda_bucket.

C.A logical table will persist the physical plan to the Hive Metastore in the Databricks control plane.

D.An managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.

E.A managed table will be created in the DBFS root storage container.

Answer: D

Explanation:

An managed table will be created in the storage container mounted to /mnt/finance_eda_bucket.

Reference:

https://docs.databricks.com/en/data-governance/unity-catalog/create-schemas.html#language-SQL

Question: 46 **Deep Dumps**

Although the Databricks Utilities Secrets module provides tools to store sensitive credentials and avoid accidentally displaying them in plain text users should still be careful with which credentials are stored here and which users have access to using these secrets.
Which statement describes a limitation of Databricks Secrets?

A.Because the SHA256 hash is used to obfuscate stored secrets, reversing this hash will display the value in plain text.

B.Account administrators can see all secrets in plain text by logging on to the Databricks Accounts console.

C.Secrets are stored in an administrators-only table within the Hive Metastore; database administrators have permission to query this table by default.

D.Iterating through a stored secret and printing each character will display secret contents in plain text.

E.The Databricks REST API can be used to list secrets in plain text if the personal access token has proper credentials.

Answer: B

Explanation:

The correct answer is B: Account administrators can see all secrets in plain text by logging on to the Databricks Accounts console. Here's why:

Databricks Secrets are designed to protect sensitive information by storing them securely in a backend system (e.g., Azure Key Vault, AWS Secrets Manager). While Databricks obscures these secrets within the Databricks workspace itself, making them difficult for regular users to access directly, a key limitation arises from the access granted to account administrators.

Account administrators typically possess elevated privileges to manage the entire Databricks account, including workspace settings, user permissions, and security configurations. This level of access can include the ability to view secrets in plaintext through the Databricks Accounts console or through API calls utilizing their administrative credentials. This is because, to effectively manage and troubleshoot the account, administrators sometimes require the ability to audit or verify security configurations related to secret usage.

Options A, C, D, and E are incorrect:

A is incorrect: Databricks Secrets use encryption, not hashing (like SHA256), to protect secrets. Encryption is reversible with the correct key, while a hash is not.
C is incorrect: Secrets are not stored in the Hive Metastore. They're stored in a secure secret store (like Azure Key Vault or AWS Secrets Manager). The Hive Metastore stores metadata about data tables, not sensitive credentials.
D is incorrect: Databricks is designed not to print secrets in plain text even when iterating. Trying to directly access the secret will return a redacted value or an error, preventing accidental exposure.
E is incorrect: The Databricks REST API will not return secrets in plain text, even with proper credentials. It only provides operations to manage the secret's existence and scope, not to reveal its content.

In summary, while Databricks Secrets offer a layer of security, the principle of least privilege applies. The fact that account administrators can view secrets in plaintext necessitates carefully managing who holds those administrative roles and implementing robust auditing practices.

Authoritative Links:

Databricks Secrets: https://docs.databricks.com/security/secrets/index.html
Azure Key Vault: https://learn.microsoft.com/en-us/azure/key-vault/general/basic-concepts
AWS Secrets Manager: https://aws.amazon.com/secrets-manager/

Question: 47 **Deep Dumps**

What statement is true regarding the retention of job run history?

A.It is retained until you export or delete job run logs

B.It is retained for 30 days, during which time you can deliver job run logs to DBFS or S3

C.It is retained for 60 days, during which you can export notebook run results to HTML

D.It is retained for 60 days, after which logs are archived

E.It is retained for 90 days or until the run-id is re-used through custom run configuration

Answer: C

Explanation:

The correct answer is C: It is retained for 60 days, during which you can export notebook run results to HTML.

Here's a detailed justification:

Databricks retains the history of job runs for a specific period to enable monitoring, debugging, and auditing of past executions. Understanding this retention policy is crucial for managing job run data and archiving necessary information before it's purged. While logs are critical, Databricks provides options to export various artifacts, including notebook run results.

Option A is incorrect because Databricks automatically manages job run retention; it's not solely contingent on manual export or deletion. Option B is incorrect because the retention period exceeds 30 days. Option D is partially correct about the 60-day period but incorrect in that logs are archived. Instead they are purged. Option E is incorrect because the 90-day retention claim is inaccurate, and run-id re-use doesn't directly affect retention.

Option C is correct because Databricks indeed retains job run history for 60 days. During this period, users can export the results of notebook executions, often in HTML format, for archival or analysis. This allows for detailed inspection of the notebook's output, including visualizations and data, at a later time. The ability to export results is particularly beneficial for long-term monitoring or compliance purposes. This option is also the only one mentioning specifically notebook run results.

Further research can be conducted using the official Databricks documentation:

Databricks Job Monitoring: https://docs.databricks.com/workflows/jobs/index.html#monitor-jobs
Databricks Job API: https://docs.databricks.com/api/workspace/jobs (while not directly addressing retention period, it shows the API interface for interacting with job runs, implying the existence of a retention mechanism).

Question: 48 **Deep Dumps**

A data engineer, User A, has promoted a new pipeline to production by using the REST API to programmatically create several jobs. A DevOps engineer, User B, has configured an external orchestration tool to trigger job runs through the REST API. Both users authorized the REST API calls using their personal access tokens.
Which statement describes the contents of the workspace audit logs concerning these events?

A.Because the REST API was used for job creation and triggering runs, a Service Principal will be automatically used to identify these events.

B.Because User B last configured the jobs, their identity will be associated with both the job creation events and the job run events.

C.Because these events are managed separately, User A will have their identity associated with the job creation events and User B will have their identity associated with the job run events.

D.Because the REST API was used for job creation and triggering runs, user identity will not be captured in the audit logs.

E.Because User A created the jobs, their identity will be associated with both the job creation events and the job run events.

Answer: C

Explanation:

Here's a detailed justification for why option C is the most accurate description of the Databricks audit logs in the given scenario:

The core concept here is that Databricks audit logs record actions based on the identity that initiated the action. In this case, two distinct actions are happening: job creation and job run triggering.

User A created the jobs programmatically using the REST API and their personal access token. Therefore, the audit logs will record User A's identity as the entity that performed the job creation actions. This is a direct consequence of the REST API using User A's personal access token to authenticate the requests. Databricks associates actions with the user associated with the token used.

Subsequently, User B configured an external orchestration tool (also using the REST API and their personal access token) to trigger job runs. When the orchestration tool executes and makes REST API calls to trigger the job runs, it's doing so under User B's identity. This is because User B's personal access token is being used for authentication.

Therefore, the audit logs will separately record User B's identity as the entity that triggered the job run events. It's crucial to remember that Databricks auditing aims to track the specific user or service principal responsible for each discrete action. Since job creation and run triggering are separate actions initiated by different authenticated entities, they are logged separately.

Option A is incorrect because a Service Principal would need to be explicitly configured and used for authentication; simply using the REST API does not automatically trigger Service Principal usage. Option B is incorrect because while User B configured the job runs, they did not create the jobs themselves. Option D is incorrect as user identity is captured in the audit logs when using personal access tokens. Option E is also incorrect because User A only created the jobs and did not trigger the runs. The audit logs track distinct actions separately.

Here are some relevant links for further research:

Databricks Audit Logging: https://docs.databricks.com/administration-guide/account-settings/audit-logs.html
Databricks REST API Authentication: https://docs.databricks.com/dev-tools/api/latest/authentication.html

Question: 49 **Deep Dumps**

A user new to Databricks is trying to troubleshoot long execution times for some pipeline logic they are working on. Presently, the user is executing code cell-by-cell, using display() calls to confirm code is producing the logically correct results as new transformations are added to an operation. To get a measure of average time to execute, the user is running each cell multiple times interactively.
Which of the following adjustments will get a more accurate measure of how code is likely to perform in production?

A.Scala is the only language that can be accurately tested using interactive notebooks; because the best performance is achieved by using Scala code compiled to JARs, all PySpark and Spark SQL logic should be refactored.

B.The only way to meaningfully troubleshoot code execution times in development notebooks Is to use production-sized data and production-sized clusters with Run All execution.

C.Production code development should only be done using an IDE; executing code against a local build of open source Spark and Delta Lake will provide the most accurate benchmarks for how code will perform in production.

D.Calling display() forces a job to trigger, while many transformations will only add to the logical query plan; because of caching, repeated execution of the same logic does not provide meaningful results.

E.The Jobs UI should be leveraged to occasionally run the notebook as a job and track execution time during incremental code development because Photon can only be enabled on clusters launched for scheduled jobs.

Answer: B

Explanation:

The most accurate measurement of production performance is achieved by mimicking production conditions as closely as possible during testing. This includes using data volumes and cluster configurations that are representative of the production environment. Let's analyze why the other options are less optimal. Option A is incorrect because it unnecessarily limits the user to Scala. PySpark and Spark SQL are perfectly viable and performant options in Databricks. Option C proposes using a local build of open-source Spark and Delta Lake, which will likely have a different configuration and environment compared to the Databricks runtime, leading to inaccurate performance assessments. Option D correctly identifies the impact of display() and caching, but it doesn't offer a comprehensive solution for accurate performance testing. Option E suggests using the Jobs UI, but it falsely claims Photon can only be enabled for scheduled jobs. Photon is available for interactive clusters as well.

Therefore, option B is the best choice. "Run All" ensures the entire pipeline is executed, revealing the performance bottlenecks that cell-by-cell execution might mask. Utilizing production-sized data accurately reflects the system's behavior under realistic loads. Production-sized clusters ensure sufficient resources are allocated, avoiding artificial constraints that might occur on smaller development clusters. Mimicking production settings allows for reliable benchmarking.

For more information on Databricks performance tuning, refer to the official Databricks documentation:

Databricks Performance Tuning
Monitor Databricks Workloads

Question: 50 **Deep Dumps**

A production cluster has 3 executor nodes and uses the same virtual machine type for the driver and executor. When evaluating the Ganglia Metrics for this cluster, which indicator would signal a bottleneck caused by code executing on the driver?

A.The five Minute Load Average remains consistent/flat
B.Bytes Received never exceeds 80 million bytes per second
C.Total Disk Space remains constant
D.Network I/O never spikes
E.Overall cluster CPU utilization is around 25%

Answer: D

Explanation:

The correct answer is **D. Network I/O never spikes**. Here's a detailed justification:

A bottleneck on the driver node, particularly due to code execution, manifests as increased network activity. The driver orchestrates tasks, manages state, and communicates with executors. Heavy driver-side computations often involve fetching data, aggregating results, or coordinating complex operations that necessitate frequent network interactions.

If the driver is struggling with computations, it may not be able to keep up with sending instructions or receiving results from the executors quickly enough. This causes the network I/O to plateau, failing to reflect the actual processing potential of the cluster. The driver essentially becomes a choke point, limiting overall cluster throughput. A lack of network I/O spikes, while executors are otherwise idle (as potentially inferred from low overall CPU), suggests the driver isn't sending or receiving data at its optimal rate.

Option A is incorrect because a consistent load average doesn't specifically point to a driver bottleneck. It could indicate overall system load, regardless of where the bottleneck resides. Option B is incorrect because a constant Bytes Received rate might indicate limitations on another part of the infrastructure rather than the

driver node. Option C about Total Disk Space remaining constant doesn't directly relate to a driver-side CPU bottleneck. Option E, low overall CPU utilization, is partially relevant because a driver bottleneck prevents executors from being fully utilized, but it's not the most direct indicator. Network I/O provides a more precise signal about the driver's ability to communicate with the executors.

Therefore, the lack of network I/O spikes is the strongest indicator of a driver-side computational bottleneck, specifically when other metrics (implicitly or explicitly) suggest the cluster has more compute capacity than is currently being leveraged. This indicates that the bottleneck is not in processing but data movement, which is usually managed by the driver.

For more information, consult the following resources:

Apache Spark Documentation on Monitoring: Look for details about monitoring metrics like network I/O and CPU utilization. While the question specifically talks about Ganglia, the principles are similar. You can find documentation about monitoring these metrics using Spark's built-in UI and external monitoring tools.https://spark.apache.org/docs/latest/monitoring.html
Databricks Documentation on Cluster Performance: Check the Databricks documentation on troubleshooting cluster performance issues, which often includes guidance on identifying driver bottlenecks and network limitations.https://docs.databricks.com/en/clusters/

Question: 51 **Deep Dumps**

Where in the Spark UI can one diagnose a performance problem induced by not leveraging predicate push-down?

A.In the Executor’s log file, by grepping for "predicate push-down"

B.In the Stage’s Detail screen, in the Completed Stages table, by noting the size of data read from the Input column

C.In the Storage Detail screen, by noting which RDDs are not stored on disk

D.In the Delta Lake transaction log. by noting the column statistics

E.In the Query Detail screen, by interpreting the Physical Plan

Answer: E

Explanation:

The correct answer is **E. In the Query Detail screen, by interpreting the Physical Plan.** Here's why:

Predicate pushdown is a performance optimization technique in Spark (and other database systems) where filtering operations (predicates) are applied as early as possible in the query execution plan, ideally at the data source level. This reduces the amount of data that needs to be read and processed, leading to significant performance improvements.

The Spark UI's Query Detail screen is the most appropriate place to diagnose a lack of predicate pushdown because it displays the **Physical Plan** of your query. The Physical Plan visualizes how Spark intends to execute your query.

Why the Physical Plan matters: By examining the Physical Plan, you can see exactly where filtering operations are occurring. If predicate pushdown is working correctly, you'll see filters being applied before data is read from the data source (e.g., before a Parquet scan or a Delta table read). If it's not working, you'll see that Spark is reading a large amount of data and then applying the filter, indicating a performance bottleneck.

Example: Imagine you are querying a large table with a WHERE clause limiting the data to a specific date range. If predicate pushdown is enabled and effective, the Physical Plan will show the data source (e.g.,

Parquet file) being filtered before it's read into Spark. If it's not working, you'll see Spark reading the entire Parquet file and then applying the date filter, which is less efficient.

Let's address why the other options are less suitable:

A. Executor Logs: While executor logs contain valuable information, grepping for "predicate push-down" is unlikely to provide a clear indication of whether it's effectively working or not. You won't see the Physical Plan.
B. Stage Detail Screen: The Stage Detail screen shows metrics about stages of execution, including data read. A large input size might suggest a problem, but it doesn't pinpoint the reason (lack of predicate pushdown) as directly as the Physical Plan does. There may be other reasons to read a large input such as large schemas.
C. Storage Detail Screen: The Storage Detail screen focuses on RDD persistence. While caching can improve performance, it doesn't directly relate to predicate pushdown. The core issue is the amount of input to spark not caching behaviour.
D. Delta Lake Transaction Log: The Delta Lake transaction log primarily records metadata about changes to the Delta table. It does contain column statistics which enable predicate pushdown, but it doesn't directly show whether predicate pushdown actually occurred in a specific query. While the metadata supports predicate pushdown, seeing it active is different.

Therefore, the Query Detail screen and its Physical Plan provide the clearest and most direct way to diagnose performance problems caused by a lack of predicate pushdown. You can see visually whether filters are being applied early in the process, which is crucial for optimization.

Authoritative Links:

Apache Spark Documentation on Understanding the Query Plan: https://spark.apache.org/docs/latest/sql-performance-tuning.html (Look for sections explaining how to analyze the query plan.)
Databricks Documentation on Predicate Pushdown: (Search Databricks documentation for "predicate pushdown" for specific details on how it works with Databricks and Delta Lake; you'll often find it within Delta Lake performance tuning documentation.) Databricks documentation is often proprietary and requires an account, so be wary of linking directly to specific pages that might require authentication to view.

Question: 52 **Deep Dumps**

Review the following error traceback:

```
---------------------------------------------------------------------------
AnalysisException                         Traceback (most recent call last)
<command-3293767849433948> in <module>
----> 1 display(df.select(3*"heartrate"))

/databricks/spark/python/pyspark/sql/dataframe.py in select(self, *cols)
   1690         [Row(name='Alice', age=12), Row(name='Bob', age=15)]
   1691         """
-> 1692         jdf = self._jdf.select(self._jcols(*cols))
   1693         return DataFrame(jdf, self.sql_ctx)
   1694

/databricks/spark/python/lib/py4j-0.10.9-src.zip/py4j/java_gateway.py in __call__(self, *args)
   1302
   1303         answer = self.gateway_client.send_command(command)
-> 1304         return_value = get_return_value(
   1305             answer, self.gateway_client, self.target_id, self.name)
   1306

/databricks/spark/python/pyspark/sql/utils.py in deco(*a, **kw)
    121                 # Hide where the exception came from that shows a non-Pythonic
    122                 # JVM exception message.
--> 123                 raise converted from None
    124             else:
    125                 raise

AnalysisException: cannot resolve '`heartrateheartrateheartrate`' given input columns:
[spark_catalog.database.table.device_id, spark_catalog.database.table.heartrate,
spark_catalog.database.table.mrn, spark_catalog.database.table.time];
'Project ['heartrateheartrateheartrate]
+- SubqueryAlias spark_catalog.database.table
   +- Relation[device_id#75L,heartrate#76,mrn#77L,time#78] parquet
```

Which statement describes the error being raised?

A.The code executed was PySpark but was executed in a Scala notebook.
B.There is no column in the table named heartrateheartrateheartrate
C.There is a type error because a column object cannot be multiplied.
D.There is a type error because a DataFrame object cannot be multiplied.
E.There is a syntax error because the heartrate column is not correctly identified as a column.

Answer: B

Explanation:

It's B, there is no column with that name.

Question: 53 **Deep Dumps**

Which distribution does Databricks support for installing custom Python code packages?

A.sbt
B.CRANC. npm
D.Wheels
E.jars

Answer: D

Explanation:

The correct answer is indeed D, Wheels. Here's a detailed justification:

Databricks, a leading cloud-based data engineering platform built on Apache Spark, leverages Python extensively. To extend the functionality of Databricks clusters with custom or third-party Python code, users need a mechanism to package and distribute these code packages. Databricks supports the installation of Python packages using the pip package installer, the standard package installer for Python. Pip relies on the Wheel format for distributing Python packages.

Wheels (extension .whl) are a built-package format for Python. They are designed to be easily installed without requiring compilation, making them faster and more reliable compared to source distributions. This is crucial in a cloud environment like Databricks where minimizing deployment time and ensuring consistency are paramount. Wheels contain all the necessary files and metadata for a Python package in a pre-built format, ready for installation.

Options A (sbt), B (CRAN), C (npm), and E (jars) are incorrect because they pertain to different package management ecosystems. sbt is a build tool for Scala projects, CRAN is the Comprehensive R Archive Network for R packages, npm is the Node Package Manager for JavaScript packages, and jars are Java Archive files used for distributing Java code. While Databricks supports Scala, R, Java, and JavaScript through various means, when it comes to Python, Wheels are the primary supported distribution format.

Therefore, to reliably install custom Python libraries and dependencies on Databricks clusters, using the Wheel format is the recommended and supported approach, ensuring compatibility and efficient deployment. Databricks provides mechanisms to install Wheels directly from DBFS (Databricks File System), cloud storage, or through a custom package index.

For further research, you can refer to the official Databricks documentation on libraries and package management:

Databricks documentation on Libraries: https://docs.databricks.com/libraries/index.html
Pip documentation: https://pip.pypa.io/en/stable/
Wheel documentation: https://packaging.python.org/en/latest/specifications/binary-distribution-format/

Question: 54 **Deep Dumps**

Which Python variable contains a list of directories to be searched when trying to locate required modules?

A.importlib.resource_path
B.sys.path
C.os.path
D.pypi.path
E.pylib.source

Answer: B

Explanation:

The correct answer is B, sys.path. This variable, accessible through the sys module in Python, holds a list of directory paths that the Python interpreter searches when importing modules. When you execute an import statement, Python iterates through the paths specified in sys.path in the order they appear, looking for the module file (either a .py file, a .pyc file, or a compiled extension).

Option A, importlib.resource_path, is related to accessing resources packaged within a module, not the general

module search path. Option C, os.path, is a module containing functions for manipulating file paths, but it doesn't hold the list of directories searched for modules. Option D, pypi.path, is incorrect because pypi refers to the Python Package Index, which is a repository of software packages, and doesn't have a direct path attribute. Option E, pylib.source, is incorrect as pylib isn't a standard python module, and it doesn't contain information on module source paths.

sys.path is crucial for managing module resolution in Python environments, especially within cloud environments like Databricks where custom libraries or modules may be stored in non-standard locations. Modifying sys.path allows you to import modules from specific directories, which can be useful for organizing project dependencies and ensuring that the correct versions of libraries are used. In Databricks, you often deal with managing various dependencies to support data engineering pipelines, and sys.path enables flexibility in locating and importing these required modules.

For more information, refer to the Python documentation on the sys module and the import statement.https://docs.python.org/3/library/sys.htmlhttps://docs.python.org/3/reference/import.html

Question: 55 **Deep Dumps**

Incorporating unit tests into a PySpark application requires upfront attention to the design of your jobs, or a potentially significant refactoring of existing code.
Which statement describes a main benefit that offset this additional effort?

A.Improves the quality of your data
B.Validates a complete use case of your application
C.Troubleshooting is easier since all steps are isolated and tested individually
D.Yields faster deployment and execution times
E.Ensures that all steps interact correctly to achieve the desired end result

Answer: C

Explanation:

The correct answer is C: "Troubleshooting is easier since all steps are isolated and tested individually." Here's why:

Unit testing inherently focuses on isolating individual components (functions, classes, or small units of code) within a larger application and verifying their correct behavior independently. This is a fundamental principle of software engineering. In a PySpark application, this translates to testing individual transformations, data cleansing operations, or aggregation steps.

By isolating and testing each step, you can quickly identify the exact location where an error occurs. If a unit test fails, it pinpoints the specific piece of code responsible, reducing the scope of your investigation. This contrasts sharply with debugging a complex, monolithic Spark job where tracing errors through multiple interconnected transformations can be time-consuming and difficult.

Options A, B, D, and E are less directly related to the main benefit that offsets the effort of incorporating unit tests. While unit tests can indirectly contribute to data quality (A) by ensuring individual data transformations are correct, that's not the primary benefit. Validating a complete use case (B) is more the domain of integration or end-to-end tests. Unit tests don't inherently yield faster deployment times (D), although they can indirectly improve code quality which might lead to more stable deployments. Ensuring steps interact correctly (E) is again more the domain of integration testing; unit tests focus on individual components, not their interactions.

The effort in upfront design or refactoring to enable unit testing in PySpark is compensated by the significantly improved ease of troubleshooting. When issues arise (and they inevitably do in complex data pipelines), unit tests provide a fast and targeted way to locate and fix the root cause, saving valuable debugging time. This makes the entire development and maintenance process more efficient.

For further reading, explore these resources:

Spark Testing Base: https://github.com/holdenk/spark-testing-base (A library facilitating unit testing of Spark applications)
Testing in PySpark: Search for "PySpark unit testing" or "testing Spark applications" online for various blog posts and tutorials.

In summary, the major advantage of incorporating unit tests into a PySpark application is the significant reduction in troubleshooting time and effort due to the isolation and independent verification of individual code components.

Question: 56 **Deep Dumps**

Which statement describes integration testing?

A.Validates interactions between subsystems of your application
B.Requires an automated testing framework
C.Requires manual intervention
D.Validates an application use case
E.Validates behavior of individual elements of your application

Answer: A

Explanation:

The correct answer is A: Validates interactions between subsystems of your application.

Integration testing focuses on verifying the communication and data flow between different modules or components of a software system. It aims to uncover defects that arise when independently developed units are combined. These defects often relate to interface issues, data inconsistencies, or unexpected interactions.

Option B is incorrect because integration testing doesn't necessarily require an automated framework, though automation significantly improves its efficiency and repeatability, particularly in complex systems. Manual integration testing is feasible for smaller systems.

Option C is incorrect because while manual intervention can be part of integration testing, it's not a requirement. Automated tests can be designed to run without manual intervention, providing quicker feedback.

Option D is incorrect because validating an application use case is more closely aligned with end-to-end or system testing. End-to-end testing ensures the entire application works as expected from start to finish, mimicking real-world user scenarios.

Option E is incorrect because validating the behavior of individual elements of the application describes unit testing. Unit testing focuses on testing individual functions, classes, or modules in isolation.

In essence, integration testing bridges the gap between unit testing (individual components) and system testing (the entire application). It validates that the different parts of the system work together harmoniously. A strong integration testing strategy is critical to building reliable and robust software.

For further reading on Integration Testing, refer to the following resources:

Guru99: https://www.guru99.com/integration-testing.html
BrowserStack: https://www.browserstack.com/guide/integration-testing

Question: 57 **Deep Dumps**

Which REST API call can be used to review the notebooks configured to run as tasks in a multi-task job?

A./jobs/runs/list
B./jobs/runs/get-output
C./jobs/runs/get
D./jobs/get
E./jobs/list

Answer: D

Explanation:

The correct REST API call to review notebooks configured as tasks within a multi-task Databricks job is /jobs/get. This is because the /jobs/get endpoint retrieves the full details of a specified job, including the configuration of each task within the job. That configuration would include the notebook path or other task details, allowing you to review which notebooks are used.

Option A, /jobs/runs/list, is used to list the historical runs of a job, not the job's configuration. Option B, /jobs/runs/get-output, is designed to retrieve the output of a specific job run, not the underlying task definitions. Similarly, /jobs/runs/get (Option C) retrieves information about a specific job run, not the job configuration itself. Finally, /jobs/list (Option E) only provides a listing of all jobs, without detailed task configurations for individual jobs. Therefore, to inspect the configuration of a job, including the notebooks used in tasks, /jobs/get is the only appropriate REST API endpoint.

Authoritative documentation can be found at Databricks Jobs API. Specifically, examine the section detailing the GET /jobs/ job_id endpoint. This provides the specifications for extracting job configurations, which will contain the notebook definitions.

Question: 58 **Deep Dumps**

A Databricks job has been configured with 3 tasks, each of which is a Databricks notebook. Task A does not depend on other tasks. Tasks B and C run in parallel, with each having a serial dependency on task A.
If tasks A and B complete successfully but task C fails during a scheduled run, which statement describes the resulting state?

A.All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.

B.All logic expressed in the notebook associated with tasks A and B will have been successfully completed; any changes made in task C will be rolled back due to task failure.

C.All logic expressed in the notebook associated with task A will have been successfully completed; tasks B and C will not commit any changes because of stage failure.

D.Because all tasks are managed as a dependency graph, no changes will be committed to the Lakehouse until ail tasks have successfully been completed.

E.Unless all tasks complete successfully, no changes will be committed to the Lakehouse; because task C failed, all commits will be rolled back automatically.

Answer: A

Explanation:

The correct answer is **A. All logic expressed in the notebook associated with tasks A and B will have been successfully completed; some operations in task C may have completed successfully.**

Here's a detailed justification:

Databricks Jobs execute tasks based on a defined dependency graph. In this scenario, Task A runs independently, and its successful completion is a prerequisite for Tasks B and C. Since Tasks B and C run in parallel after Task A, the successful completion of A means all operations within its associated notebook have been executed and committed. Task B also completes successfully, indicating its notebook's logic was entirely executed and its results committed.

Task C failing doesn't automatically roll back the successful operations of Tasks A and B. Databricks doesn't employ an all-or-nothing, atomic transaction model across independent job tasks by default. While Databricks supports ACID transactions within a given task, it doesn't guarantee that a failure in one task will roll back the results of other, already completed tasks in a job.

Task C's failure indicates that the notebook associated with it encountered an error during execution. Some operations within the task C notebook might have been executed and have produced side effects before the point of failure. Therefore, the statement "some operations in task C may have completed successfully" is accurate.

Options B, C, D, and E are incorrect because they assume a level of atomicity and rollback capability across independent job tasks that Databricks doesn't provide by default. While transaction control is possible within a Databricks task using techniques like BEGIN TRANSACTION, COMMIT, and ROLLBACK in Spark SQL or similar approaches, a simple task failure doesn't inherently trigger a rollback of successful tasks that depend on it.

In summary, the partial completion of Task C, along with the successful completion of tasks A and B, demonstrates how independent tasks within a Databricks job can execute and commit independently of each other's success, which means option A is the only possibility.

Further research:

Databricks Jobs: https://docs.databricks.com/workflows/jobs/index.html
Delta Lake ACID Transactions: https://docs.databricks.com/delta/transaction-isolation.html

Question: 59 **Deep Dumps**

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_stor
USING DELTA
LOCATION "/mnt/prod/sales_by_store"
```

Realizing that the original query had a typographical error, the below code was executed:
ALTER TABLE prod.sales_by_stor RENAME TO prod.sales_by_store
Which result will occur after running the second command?

A.The table reference in the metastore is updated and no data is changed.

B.The table name change is recorded in the Delta transaction log.

C.All related files and metadata are dropped and recreated in a single ACID transaction.

D.The table reference in the metastore is updated and all data files are moved.

E.A new Delta transaction log Is created for the renamed table.

Answer: A

Explanation:

A. The table reference in the metastore is updated and no data is changed.

When an ALTER TABLE ... RENAME TO ... command is executed on a Delta Lake table, it is a metadata-only operation within the metastore (e.g., Hive Metastore or Unity Catalog). The physical data files and their storage location in cloud object storage are not moved or altered. The table's new name is updated as a pointer to the existing data location in the metastore, which is then recorded in the Delta transaction log.

Question: 60 **Deep Dumps**

The data engineering team maintains a table of aggregate statistics through batch nightly updates. This includes total sales for the previous day alongside totals and averages for a variety of time periods including the 7 previous days, year-to-date, and quarter-to-date. This table is named store_saies_summary and the schema is as follows:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT,  total_sales_ytd FLOAT,
avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT, avg_daily_sales_7d
FLOAT, updated TIMESTAMP
```

The table daily_store_sales contains all the information needed to update store_sales_summary. The schema for this table is: store_id INT, sales_date DATE, total_sales FLOAT
If daily_store_sales is implemented as a Type 1 table and the total_sales column might be adjusted after manual data auditing, which approach is the safest to generate accurate reports in the store_sales_summary table?

A.Implement the appropriate aggregate logic as a batch read against the daily_store_sales table and overwrite the store_sales_summary table with each Update.

B.Implement the appropriate aggregate logic as a batch read against the daily_store_sales table and append new rows nightly to the store_sales_summary table.

C.Implement the appropriate aggregate logic as a batch read against the daily_store_sales table and use upsert logic to update results in the store_sales_summary table.

D.Implement the appropriate aggregate logic as a Structured Streaming read against the daily_store_sales table and use upsert logic to update results in the store_sales_summary table.

E.Use Structured Streaming to subscribe to the change data feed for daily_store_sales and apply changes to the aggregates in the store_sales_summary table with each update.

Answer: E

Explanation:

Use Structured Streaming to subscribe to the change data feed for daily_ store_ sales and apply changes to the aggregates in the store_ sales_ summary table with each update.

Question: 61 **Deep Dumps**

A member of the data engineering team has submitted a short notebook that they wish to schedule as part of a larger data pipeline. Assume that the commands provided below produce the logically correct results when run as presented.

Cmd 1

```
rawDF = spark.table("raw_data")
```

Cmd 2

```
rawDF.printSchema()
```

Cmd 3

```
flattenedDF = rawDF.select("*", "values.*")
```

Cmd 4

```
finalDF = flattenedDF.drop("values")
```

Cmd 5

```
finalDF.explain()
```

Cmd 6

```
display(finalDF)
```

Cmd 7

```
finalDF.write.mode("append").saveAsTable("flat_data")
```

Which command should be removed from the notebook before scheduling it as a job?

A.Cmd 2
B.Cmd 3
C.Cmd 4
D.Cmd 5
E.Cmd 6

Answer: E

Explanation:

Correct answer is E:Cmd 6.

When scheduling a Databricks notebook as a job, it's generally recommended to remove or modify commands that involve displaying output, such as using the display() function. Displaying data using display() is an

interactive feature designed for exploration and visualization within the notebook interface and may not work well in a production job context.

The finalDF.explain() command, which provides the execution plan of the DataFrame transformations and actions, is often useful for debugging and optimizing queries. While it doesn't display interactive visualizations like display(), it can still be informative for understanding how Spark is executing the operations on your DataFrame.

Question: 62 **Deep Dumps**

The business reporting team requires that data for their dashboards be updated every hour. The total processing time for the pipeline that extracts transforms, and loads the data for their pipeline runs in 10 minutes.

Assuming normal operating conditions, which configuration will meet their service-level agreement requirements with the lowest cost?

A.Manually trigger a job anytime the business reporting team refreshes their dashboards
B.Schedule a job to execute the pipeline once an hour on a new job cluster
C.Schedule a Structured Streaming job with a trigger interval of 60 minutes
D.Schedule a job to execute the pipeline once an hour on a dedicated interactive cluster
E.Configure a job that executes every time new data lands in a given directory

Answer: B

Explanation:

The correct answer is B. Here's why:

Service-Level Agreement (SLA) Compliance: The business requirement is to update the dashboards hourly. Option B directly addresses this by scheduling a job to run every hour.

Cost Efficiency:

Option B (New Job Cluster): This approach creates a new Databricks cluster specifically for each hourly run, executes the pipeline, and then shuts down the cluster. Databricks clusters are billed by the second, but only when running. Since the pipeline only takes 10 minutes, the cluster will only run for that duration, minimizing cost.
Option A (Manual Trigger): Requires manual intervention, which is unsustainable and error-prone. It is also difficult to enforce the hourly SLA.
Option C (Structured Streaming): Structured Streaming is designed for continuous data ingestion and processing. While you could use a trigger interval of 60 minutes, it may not be ideal for batch-oriented pipelines. Additionally, it's usually associated with a long-running cluster, potentially leading to higher costs.
Option D (Dedicated Interactive Cluster): Keeps a Databricks cluster running 24/7. This is the most expensive option because you pay for the cluster even when it's idle for 50 minutes each hour.
Option E (Trigger on Data Arrival): The data arrival pattern may not be consistent with the hourly requirement. If data arrives more or less frequently than once an hour, the dashboards will not be updated as required.

In summary, scheduling a Databricks job to execute the pipeline once an hour on a new job cluster (Option B) meets the SLA requirements at the lowest cost because it only runs the cluster when processing data, avoiding unnecessary idle time charges.

Further research:

Databricks Jobs: https://docs.databricks.com/jobs.html
Databricks Cost Management: https://docs.databricks.com/administration-guide/account-settings/cost-management.html
Spark Structured Streaming: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

Question: 63 **Deep Dumps**

A Databricks SQL dashboard has been configured to monitor the total number of records present in a collection of Delta Lake tables using the following query pattern:

SELECT COUNT (*) FROM table -

Which of the following describes how results are generated each time the dashboard is updated?

A.The total count of rows is calculated by scanning all data files
B.The total count of rows will be returned from cached results unless REFRESH is run
C.The total count of records is calculated from the Delta transaction logs
D.The total count of records is calculated from the parquet file metadata
E.The total count of records is calculated from the Hive metastore

Answer: C

Explanation:

The correct answer is C: "The total count of records is calculated from the Delta transaction logs."

Here's a detailed justification:

Delta Lake enhances Parquet data files with a transaction log (Delta log). This log meticulously records every change made to the Delta table, including additions, deletions, and modifications. Crucially, it also tracks metadata about the table, including the total number of records.

When a SELECT COUNT(*) FROM table query is executed against a Delta Lake table, Databricks can efficiently retrieve the row count from the Delta log, rather than scanning the entire dataset. This optimization is possible because the Delta log maintains a consistent and up-to-date record of the table's metadata, including the total row count after each transaction. This is significantly faster than scanning all the underlying Parquet files, which would be necessary in a traditional data lake.

The other options are incorrect because:

A. The total count of rows is calculated by scanning all data files: This is inefficient and defeats the purpose of Delta Lake's optimizations. Delta Lake aims to avoid full table scans for simple aggregate queries like COUNT(*).

B. The total count of rows will be returned from cached results unless REFRESH is run: While caching can improve performance, the initial count must still come from somewhere. The Delta log is the source. Also, dashboards typically refresh periodically, so relying solely on cached results could lead to outdated information. While result caching exists within Databricks SQL, it doesn't invalidate the core logic of how COUNT(*) is optimized with Delta Lake.

D. The total count of records is calculated from the parquet file metadata: Parquet file metadata typically stores statistics about the data within that specific file, not the total number of records in the table. While Parquet stores row counts in the file footer, aggregating these counts from all files is still more expensive and

less efficient than retrieving the count from the Delta Log.

E. The total count of records is calculated from the Hive metastore: The Hive metastore primarily stores schema information and table locations, not detailed record counts. While integrations may extract basic stats, relying on the metastore for an accurate COUNT(*) would lack the transactional guarantees and freshness provided by the Delta Log. The Hive metastore has no understanding of delta logs' fine-grained change tracking and versioning.

Therefore, Delta Lake leverages its transaction log to provide a fast and accurate response to COUNT(*) queries, making option C the most accurate description of the process.

Here are some authoritative links for further research:

Delta Lake Documentation: ACID Transactions on Apache Spark™:
https://docs.databricks.com/delta/index.html
Understanding Delta Lake Performance: https://www.databricks.com/blog/2019/08/22/diving-into-delta-lake-unpacking-the-transaction-log.html

Question: 64 **Deep Dumps**

A Delta Lake table was created with the below query:

```
CREATE TABLE prod.sales_by_store
AS (
  SELECT *
  FROM prod.sales a
  INNER JOIN prod.store b
  ON a.store_id = b.store_id
)
```

Consider the following query:

DROP TABLE prod.sales_by_store -

If this statement is executed by a workspace admin, which result will occur?

A.Nothing will occur until a COMMIT command is executed.
B.The table will be removed from the catalog but the data will remain in storage.
C.The table will be removed from the catalog and the data will be deleted.
D.An error will occur because Delta Lake prevents the deletion of production data.
E.Data will be marked as deleted but still recoverable with Time Travel.

Answer: C

Explanation:

The table will be removed from the catalog and the data will be deleted.

Question: 65 **Deep Dumps**

Two of the most common data locations on Databricks are the DBFS root storage and external object storage mounted with dbutils.fs.mount().

Which of the following statements is correct?

A.DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.

B.By default, both the DBFS root and mounted data sources are only accessible to workspace administrators.

C.The DBFS root is the most secure location to store data, because mounted storage volumes must have full public read and write permissions.

D.Neither the DBFS root nor mounted storage can be accessed when using %sh in a Databricks notebook.

E.The DBFS root stores files in ephemeral block volumes attached to the driver, while mounted directories will always persist saved data to external storage between sessions.

Answer: A

Explanation:

The correct answer is A: DBFS is a file system protocol that allows users to interact with files stored in object storage using syntax and guarantees similar to Unix file systems.

Here's a detailed justification:

DBFS Abstraction: Databricks File System (DBFS) is an abstraction layer on top of object storage (like AWS S3 or Azure Blob Storage). It provides a file system interface to interact with data, regardless of the underlying storage. This allows users to use familiar file system commands (like ls, cp, mv) to manage data.
Unix-like Syntax: DBFS aims to provide a user experience similar to traditional Unix file systems. While not a perfect match, the basic operations and hierarchical structure are intended to be familiar.
Underlying Object Storage: The actual data is not stored on the Databricks compute instances themselves. It resides in the configured object storage. DBFS translates file system operations into object storage API calls.

Why other options are incorrect:

B: DBFS root and mounted data sources do not default to only being accessible to workspace administrators. Permissions can be configured with access control lists (ACLs), but by default, access is more open within the workspace.
C: The DBFS root is not the most secure by default. In fact, mounted storage often offers more fine-grained control over access permissions through cloud provider IAM roles and policies. Furthermore, mounted storage doesn't require full public read and write permissions. You can configure specific permissions through service principals, IAM roles, or similar mechanisms.
D: Both the DBFS root and mounted storage can be accessed when using %sh in a Databricks notebook. The shell environment has access to the DBFS CLI, allowing interaction with the file system.
E: The DBFS root does not store files in ephemeral block volumes attached to the driver. As stated earlier, it resides in object storage. Furthermore, both the DBFS root and mounted directories persist data to external storage between sessions. Data written to either location will persist after the cluster is terminated.

Supporting Links:

Databricks DBFS Documentation: https://docs.databricks.com/data/databricks-file-system.html
Databricks Access Control: https://docs.databricks.com/security/access-control/index.html

Question: 66 **Deep Dumps**

The following code has been migrated to a Databricks notebook from a legacy workload:

```
%sh
git clone https://github.com/foo/data_loader;
python ./data_loader/run.py;
mv ./output /dbfs/mnt/new_data
```

The code executes successfully and provides the logically correct results, however, it takes over 20 minutes to extract and load around 1 GB of data.

Which statement is a possible explanation for this behavior?

A.%sh triggers a cluster restart to collect and install Git. Most of the latency is related to cluster startup time.

B.Instead of cloning, the code should use %sh pip install so that the Python code can get executed in parallel across all nodes in a cluster.

C.%sh does not distribute file moving operations; the final line of code should be updated to use %fs instead.

D.Python will always execute slower than Scala on Databricks. The run.py script should be refactored to Scala.

E.%sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

Answer: E

Explanation:

%sh executes shell code on the driver node. The code does not take advantage of the worker nodes or Databricks optimized Spark.

Question: 67 **Deep Dumps**

The data science team has requested assistance in accelerating queries on free form text from user reviews. The data is currently stored in Parquet with the below schema:

item_id INT, user_id INT, review_id INT, rating FLOAT, review STRING

The review column contains the full text of the review left by the user. Specifically, the data science team is looking to identify if any of 30 key words exist in this field.

A junior data engineer suggests converting this data to Delta Lake will improve query performance.

Which response to the junior data engineer s suggestion is correct?

A.Delta Lake statistics are not optimized for free text fields with high cardinality.

B.Text data cannot be stored with Delta Lake.

C.ZORDER ON review will need to be run to see performance gains.

D.The Delta log creates a term matrix for free text fields to support selective filtering.

E.Delta Lake statistics are only collected on the first 4 columns in a table.

Answer: A

Explanation:

The correct response to the junior data engineer's suggestion is **A. Delta Lake statistics are not optimized for**

free text fields with high cardinality.

Here's a detailed justification:

Delta Lake enhances data reliability and performance on data lakes. It offers features such as ACID transactions, schema evolution, data versioning, and improved data quality. Delta Lake also automatically collects statistics on the data, like min/max values and null counts, to enable query optimization through data skipping.

However, the effectiveness of Delta Lake's statistics for query optimization is heavily influenced by the characteristics of the data. Delta Lake statistics excel when applied to columns with low cardinality (few distinct values) or when used for equality predicates in WHERE clauses. They help the query engine skip irrelevant data files, significantly speeding up queries.

In this scenario, the review column contains free-form text. This type of field has very high cardinality, meaning the column contains numerous unique values. Delta Lake does collect statistics on all columns. However the benefits of delta lake statistics for query optimization are less apparent on high cardinality columns.

Therefore, the statistics collected on the review column would likely be much less effective at enabling data skipping. The min/max values or histograms captured for such a high-cardinality column offer limited value in determining which data files contain the 30 keywords the data science team is searching for. Essentially, every data file might potentially contain one or more of these keywords, rendering data skipping less effective.

Option B is incorrect because Delta Lake can store text data. Option C is incorrect because ZORDER requires low cardinality. Option D is incorrect, Delta lake does not create a term matrix. Option E is incorrect, statistics are not limited to the first 4 columns.

For further research, consider exploring:

Databricks Delta Lake Documentation: https://docs.databricks.com/delta/index.html (Focus on statistics and data skipping)
Cardinality in Data Warehousing: Search for articles and blogs discussing the impact of cardinality on query performance and indexing techniques in data warehouses.

Question: 68 **Deep Dumps**

Assuming that the Databricks CLI has been installed and configured correctly, which Databricks CLI command can be used to upload a custom Python Wheel to object storage mounted with the DBFS for use with a production job?

A.configure
B.fs
C.jobs
D.libraries
E.workspace

Answer: D

Explanation:

The correct Databricks CLI command for uploading a custom Python Wheel to object storage (mounted with DBFS) for use with a production job is libraries. Here's why:

The libraries command directly manages libraries within Databricks, including uploading custom wheels.

Databricks libraries can be installed on clusters to provide dependencies for notebooks and jobs. Uploading a wheel using libraries ensures it's accessible and can be readily installed on the cluster(s) executing your production job. This enables reproducible and reliable execution of code.

Option A (configure) is used for configuring the Databricks CLI itself (authentication, default workspace URL, etc.), not for uploading files. Option B (fs) is for interacting directly with DBFS file system operations like copying files, listing directories, etc., but not optimized for library management. While you could use fs to upload a wheel, libraries is the more direct and appropriate approach for this use case. Option C (jobs) is used for managing Databricks Jobs, such as creating, running, and monitoring them, but not for directly handling library uploads. Option E (workspace) manages Databricks workspace objects such as notebooks, folders, and repos.

Using databricks libraries install --wheel <path_to_wheel> will upload the wheel and install it on a given cluster. This ensures that when the production job runs on that cluster, the necessary dependencies are available. This process contributes to improved environment management and deployment.

For further information, see:

Databricks CLI documentation: https://docs.databricks.com/en/dev-tools/cli/index.html
Databricks Libraries: https://docs.databricks.com/en/libraries/index.html

Question: 69 **Deep Dumps**

The business intelligence team has a dashboard configured to track various summary metrics for retail stores. This includes total sales for the previous day alongside totals and averages for a variety of time periods. The fields required to populate this dashboard have the following schema:

```
store_id INT, total_sales_qtd FLOAT, avg_daily_sales_qtd FLOAT,  total_sales_ytd
FLOAT, avg_daily_sales_ytd FLOAT, previous_day_sales FLOAT, total_sales_7d FLOAT,
avg_daily_sales_7d FLOAT, updated TIMESTAMP
```

For demand forecasting, the Lakehouse contains a validated table of all itemized sales updated incrementally in near real-time. This table, named products_per_order, includes the following fields:

```
store_id INT, order_id INT, product_id INT, quantity INT, price FLOAT,
order_timestamp TIMESTAMP
```

Because reporting on long-term sales trends is less volatile, analysts using the new dashboard only require data to be refreshed once daily. Because the dashboard will be queried interactively by many users throughout a normal business day, it should return results quickly and reduce total compute associated with each materialization.

Which solution meets the expectations of the end users while controlling and limiting possible costs?

A.Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.

B.Use Structured Streaming to configure a live dashboard against the products_per_order table within a Databricks notebook.

C.Configure a webhook to execute an incremental read against products_per_order each time the dashboard is refreshed.

D.Use the Delta Cache to persist the products_per_order table in memory to quickly update the dashboard with each query.

E.Define a view against the products_per_order table and define the dashboard against this view.

Answer: A

Explanation:

Populate the dashboard by configuring a nightly batch job to save the required values as a table overwritten with each update.

Question: 70 **Deep Dumps**

A data ingestion task requires a one-TB JSON dataset to be written out to Parquet with a target part-file size of 512 MB. Because Parquet is being used instead of Delta Lake, built-in file-sizing features such as Auto-Optimize & Auto-Compaction cannot be used.

Which strategy will yield the best performance without shuffling data?

A.Set spark.sql.files.maxPartitionBytes to 512 MB, ingest the data, execute the narrow transformations, and then write to parquet.

B.Set spark.sql.shuffle.partitions to 2,048 partitions (1TB*1024*1024/512), ingest the data, execute the narrow transformations, optimize the data by sorting it (which automatically repartitions the data), and then write to parquet.

C.Set spark.sql.adaptive.advisoryPartitionSizeInBytes to 512 MB bytes, ingest the data, execute the narrow transformations, coalesce to 2,048 partitions (1TB*1024*1024/512), and then write to parquet.

D.Ingest the data, execute the narrow transformations, repartition to 2,048 partitions (1TB* 1024*1024/512), and then write to parquet.

E.Set spark.sql.shuffle.partitions to 512, ingest the data, execute the narrow transformations, and then write to parquet.

Answer: A

Explanation:

Here's a detailed justification for why option A is the best choice, along with relevant links for further research:

The goal is to write a 1TB JSON dataset to Parquet with a target part-file size of 512MB without shuffling the data unnecessarily. Shuffling is a costly operation in Spark as it involves data movement across the network.

Option A leverages spark.sql.files.maxPartitionBytes to control the initial partitioning of the data when it's read in. By setting this to 512 MB, Spark will attempt to create partitions of approximately that size during the initial read of the JSON files. Narrow transformations (those that can be applied to each partition independently without requiring data from other partitions) can then be executed efficiently. When writing to Parquet, Spark will, ideally, create files that are close to the specified partition size. This approach minimizes shuffling as the initial partitioning is aligned with the desired output file size.

Option B involves setting spark.sql.shuffle.partitions and sorting the data. Sorting always involves a shuffle, which is what we are trying to avoid for performance reasons. Even though it aims to repartition, the shuffle makes it inefficient.

Option C uses spark.sql.adaptive.advisoryPartitionSizeInBytes which is related to Adaptive Query Execution (AQE). While AQE can dynamically adjust partition sizes, coalesce to a specific number of partitions still involves a data reorganization, although less aggressive than repartition. Coalesce is mainly for reducing number of partition not to get to the desired partition size.

Option D involves a repartition which explicitly causes a full shuffle of the data, making it the least performant option.

Option E is incorrect as setting spark.sql.shuffle.partitions to 512 will not guarantee a 512MB parquet file sizes per partition. The number of partitions only influences operations that require a shuffle.

Therefore, option A provides the most efficient approach by influencing the initial partitioning without a costly shuffle, aligning it with the desired output file size.

Key Concepts:

Shuffling: A costly operation in Spark involving data movement across executors.
Partitioning: Dividing data into smaller chunks for parallel processing.
Narrow Transformations: Operations that can be applied independently to each partition.
spark.sql.files.maxPartitionBytes: Controls the maximum number of bytes to pack into a single partition when reading files.

Authoritative Links:

Spark Configuration: https://spark.apache.org/docs/latest/configuration.html (Search for spark.sql.files.maxPartitionBytes and spark.sql.shuffle.partitions on this page)
Adaptive Query Execution (AQE): https://spark.apache.org/docs/latest/sql-performance-tuning.html#adaptive-query-execution

Question: 71 **Deep Dumps**

A junior data engineer has been asked to develop a streaming data pipeline with a grouped aggregation using DataFrame df. The pipeline needs to calculate the average humidity and average temperature for each non-overlapping five-minute interval. Incremental state information should be maintained for 10 minutes for late-arriving data.

Streaming DataFrame df has the following schema:

"device_id INT, event_time TIMESTAMP, temp FLOAT, humidity FLOAT"

Code block:

```
df._____
    .groupBy(
        window("event_time", "5 minutes").alias("time"),
        "device_id"
    )
    .agg(
        avg("temp").alias("avg_temp"),
        avg("humidity").alias("avg_humidity")
    )
    .writeStream
    .format("delta")
    .saveAsTable("sensor_avg")
```

Choose the response that correctly fills in the blank within the code block to complete this task.

A.withWatermark("event_time", "10 minutes")
B.awaitArrival("event_time", "10 minutes")
C.await("event_time + ‘10 minutes'")

D.slidingWindow("event_time", "10 minutes")

E.delayWrite("event_time", "10 minutes")

Answer: A

Explanation:

withWatermark("event_time", "10 minutes").

Question: 72 **Deep Dumps**

A data team's Structured Streaming job is configured to calculate running aggregates for item sales to update a downstream marketing dashboard. The marketing team has introduced a new promotion, and they would like to add a new field to track the number of times this promotion code is used for each item. A junior data engineer suggests updating the existing query as follows. Note that proposed changes are in bold.

Original query:

```
df.groupBy("item")
    .agg(count("item").alias("total_count"),
         mean("sale_price").alias("avg_price"))
    .writeStream
    .outputMode("complete")
    .option("checkpointLocation", "/item_agg/__checkpoint")
    .start("/item_agg")
```

Proposed query:

```
df.groupBy("item")
    .agg(count("item").alias("total_count"),
         mean("sale_price").alias("avg_price"))
    .writeStream
    .outputMode("complete")
    .option("checkpointLocation", "/item_agg/__checkpoint")
    .start("/item_agg")
```

Proposed query:

.start("/item_agg")

Which step must also be completed to put the proposed query into production?

A.Specify a new checkpointLocation

B.Increase the shuffle partitions to account for additional aggregates

C.Run REFRESH TABLE delta.'/item_agg'

D.Register the data in the "/item_agg" directory to the Hive metastore

E.Remove .option('mergeSchema', 'true') from the streaming write

Answer: A

Explanation:

A. Specify a new checkpointLocation.

To put the proposed streaming query (which involves an aggregation, specifically count("promo_code = 'NEW MEMBER'")) into production, a new checkpointLocation is required. The addition of a new aggregated field is considered an incompatible schema change in Structured Streaming when using the same checkpoint location, as the existing checkpoint metadata is tied to the previous schema. By specifying a new checkpoint location, you signal to Spark Structured Streaming to start the new job with a fresh state, thereby avoiding schema mismatch issues and ensuring proper recovery and fault tolerance in a production environment. The mergeSchema option should generally remain enabled to handle schema evolution gracefully.

Question: 73 **Deep Dumps**

A Structured Streaming job deployed to production has been resulting in higher than expected cloud storage costs. At present, during normal execution, each microbatch of data is processed in less than 3s; at least 12 times per minute, a microbatch is processed that contains 0 records. The streaming write was configured using the default trigger settings. The production job is currently scheduled alongside many other Databricks jobs in a workspace with instance pools provisioned to reduce start-up time for jobs with batch execution.

Holding all other variables constant and assuming records need to be processed in less than 10 minutes, which adjustment will meet the requirement?

A.Set the trigger interval to 3 seconds; the default trigger interval is consuming too many records per batch, resulting in spill to disk that can increase volume costs.

B.Increase the number of shuffle partitions to maximize parallelism, since the trigger interval cannot be modified without modifying the checkpoint directory.

C.Set the trigger interval to 10 minutes; each batch calls APIs in the source storage account, so decreasing trigger frequency to maximum allowable threshold should minimize this cost.

D.Set the trigger interval to 500 milliseconds; setting a small but non-zero trigger interval ensures that the source is not queried too frequently.

E.Use the trigger once option and configure a Databricks job to execute the query every 10 minutes; this approach minimizes costs for both compute and storage.

Answer: C

Explanation:

The correct answer is C because it directly addresses the core issue of excessive microbatch executions, specifically the empty ones, which lead to unnecessary API calls to the source storage and higher storage costs. The problem statement highlights that the job is processing empty microbatches at least 12 times a minute. These empty batches still trigger read operations on the source storage, incurring charges even though no actual data is being processed.

Option A is incorrect because reducing the trigger interval would actually increase the number of microbatches and API calls, exacerbating the cost issue. Option B is also incorrect; while increasing shuffle partitions might improve performance for batches with data, it won't address the problem of frequent empty batches and associated API costs. Option D suggests decreasing the trigger interval, which, as with Option A, would only increase costs by creating more frequent microbatches. Option E, while seemingly reducing compute costs, isn't practical for the business requirement of data being processed in less than 10 minutes. Trigger.Once processes data exactly one time and then shuts down which is not viable for streaming jobs.

Setting the trigger interval to 10 minutes (Option C) reduces the frequency with which the source storage is

queried. By grouping more data into each microbatch, the job makes fewer API calls, leading to significant cost savings, especially in scenarios with frequent empty batches. While this slightly increases latency, it remains within the 10-minute requirement. This approach aligns with the principle of optimizing cloud resource utilization by minimizing unnecessary I/O operations.

Relevant links for further research:

Structured Streaming Programming Guide: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html (Specifically, the "Triggers" section).
Databricks Structured Streaming: https://docs.databricks.com/structured-streaming/index.html

Question: 74 **Deep Dumps**

Which statement describes the correct use of pyspark.sql.functions.broadcast?

A.It marks a column as having low enough cardinality to properly map distinct values to available partitions, allowing a broadcast join.

B.It marks a column as small enough to store in memory on all executors, allowing a broadcast join.

C.It caches a copy of the indicated table on attached storage volumes for all active clusters within a Databricks workspace.

D.It marks a DataFrame as small enough to store in memory on all executors, allowing a broadcast join.

E.It caches a copy of the indicated table on all nodes in the cluster for use in all future queries during the cluster lifetime.

Answer: D

Explanation:

The correct answer is D because pyspark.sql.functions.broadcast(df) is specifically used to hint to Spark that the specified DataFrame df should be broadcast to all executor nodes in the cluster. This optimization is most effective when joining a relatively small DataFrame (the broadcast DataFrame) with a much larger DataFrame. By broadcasting the smaller DataFrame, Spark can avoid shuffling data from the larger DataFrame across the network during the join operation. Instead, each executor has a complete copy of the smaller DataFrame, allowing it to perform the join locally with its partition of the larger DataFrame.

Options A and B are incorrect because broadcast operates on entire DataFrames, not individual columns. It doesn't analyze column cardinality or store individual column values in memory. Option C is wrong as broadcasting doesn't involve attached storage volumes or persist data across multiple clusters. Option E is also incorrect because broadcast doesn't cache data persistently for the entire cluster lifetime or for all future queries necessarily; the broadcast effect typically lasts for the duration of the query it is applied to. Caching is separate from broadcasting. broadcast improves join performance in appropriate scenarios by distributing a small DataFrame to each executor's memory before the join, enabling faster local joins and minimizing data shuffling. This technique is a common and crucial optimization for Spark workloads that involve joining datasets of varying sizes.

Refer to the official Apache Spark documentation for more detailed information about broadcast and join optimizations:

Apache Spark SQL Performance Tuning (specifically, look for sections on Broadcast Join)
pyspark.sql.functions.broadcast

Question: 75 **Deep Dumps**

A data engineer is configuring a pipeline that will potentially see late-arriving, duplicate records.

In addition to de-duplicating records within the batch, which of the following approaches allows the data engineer to deduplicate data against previously processed records as it is inserted into a Delta table?

A.Set the configuration delta.deduplicate = true.
B.VACUUM the Delta table after each batch completes.
C.Perform an insert-only merge with a matching condition on a unique key.
D.Perform a full outer join on a unique key and overwrite existing data.
E.Rely on Delta Lake schema enforcement to prevent duplicate records.

Answer: C

Explanation:

The correct answer is C: "Perform an insert-only merge with a matching condition on a unique key." This approach leverages the MERGE operation in Delta Lake, which is ideal for handling upserts and deduplication scenarios involving potentially late-arriving or duplicate data.

Here's a detailed breakdown:

Handling Late-Arriving/Duplicate Records: The core problem is addressing records that arrive out of order or are duplicated across different processing runs.
MERGE Operation: The MERGE statement allows you to specify conditions for matching existing records in the target Delta table based on a unique key with incoming data.
Insert-Only Logic: By setting the merge condition appropriately (e.g., WHEN NOT MATCHED THEN INSERT *), the data engineer ensures that only new, unique records (those not already present based on the unique key) are inserted into the Delta table.
Deduplication: This mechanism effectively deduplicates data against previously processed records, as only records with unique keys that do not already exist in the Delta table are inserted. This provides a reliable way to deduplicate even when dealing with late-arriving data.
ACID Properties: Delta Lake's ACID properties guarantee that the merge operation is atomic, ensuring data consistency and preventing partial updates in case of failures. This is crucial for data integrity in data engineering pipelines.
Alternatives:

A (delta.deduplicate = true) is not a valid Delta Lake configuration option.
B (VACUUM) only removes old versions of data files, not duplicate data.
D (Full outer join) can be used to detect duplicates, but overwriting data can lead to data loss or unintended consequences if not handled carefully. It is also not efficient for deduplication as it requires reprocessing all records in the table every time.
E (Schema enforcement) prevents incompatible data types but does not address duplicate records.

Using MERGE with the WHEN NOT MATCHED THEN INSERT clause is the recommended approach in Databricks for deduplicating data against previously processed records in a Delta table due to its efficiency, ACID properties, and suitability for handling late-arriving data.

Authoritative Links:

Delta Lake MERGE INTO: https://docs.databricks.com/delta/delta-update.html
Databricks Delta Lake Documentation: https://docs.databricks.com/delta/index.html

Question: 76 **Deep Dumps**

A data pipeline uses Structured Streaming to ingest data from Apache Kafka to Delta Lake. Data is being stored in a bronze table, and includes the Kafka-generated timestamp, key, and value. Three months after the pipeline is deployed, the data engineering team has noticed some latency issues during certain times of the day.

A senior data engineer updates the Delta Table's schema and ingestion logic to include the current timestamp (as recorded by Apache Spark) as well as the Kafka topic and partition. The team plans to use these additional metadata fields to diagnose the transient processing delays.

Which limitation will the team face while diagnosing this problem?

A.New fields will not be computed for historic records.
B.Spark cannot capture the topic and partition fields from a Kafka source.
C.New fields cannot be added to a production Delta table.
D.Updating the table schema will invalidate the Delta transaction log metadata.
E.Updating the table schema requires a default value provided for each field added.

Answer: A

Explanation:

The correct answer is A: New fields will not be computed for historic records.

Here's a detailed justification:

The core issue revolves around the immutability of existing data. When new columns (current timestamp, Kafka topic, partition) are added to the Delta table's schema, those columns are populated for newly ingested records going forward. However, the existing records in the Delta table, representing the data from the past three months, will not be retroactively updated with these new fields. This is because Delta Lake, like most data warehousing solutions, stores data physically as it arrives. Adding columns doesn't trigger a re-processing or rewriting of historical data to fill in those new columns. Consequently, the newly added metadata will only be available for analysis from the point the schema change was implemented. This is a fundamental principle in data warehousing and lakehouse architecture.

Therefore, when the data engineering team attempts to diagnose the historical latency issues, they will find that the current_timestamp, topic, and partition fields are null or missing for the data ingested before the schema change. This limits their ability to correlate latency with specific Kafka topics, partitions, or the exact ingestion time as recorded by Spark for those older records. They can only analyze the new data being ingested after the schema change.

Option B is incorrect because Structured Streaming with Kafka can capture topic and partition information.

Option C is incorrect. Delta Lake supports schema evolution, allowing new fields to be added to production tables.

Option D is incorrect. Adding columns is a schema evolution operation that is handled by Delta Lake's transaction log and doesn't invalidate the log; it's a valid change recorded within the log.

Option E is partially correct in some cases, but not always required. Delta Lake supports adding columns without requiring a default value if the column is nullable.

Supporting Links:

Delta Lake Schema Evolution: https://docs.databricks.com/delta/delta-batch.html#schema-evolution - This documentation describes how Delta Lake handles schema evolution, including adding columns.
Structured Streaming with Kafka: https://spark.apache.org/docs/latest/structured-streaming-kafka-integration.html - This documentation showcases how Spark Structured Streaming can be used to extract details such as topic and partition information.

Question: 77 **Deep Dumps**

In order to facilitate near real-time workloads, a data engineer is creating a helper function to leverage the schema detection and evolution functionality of Databricks Auto Loader. The desired function will automatically detect the schema of the source directly, incrementally process JSON files as they arrive in a source directory, and automatically evolve the schema of the table when new fields are detected.

The function is displayed below with a blank:

```
def auto_load_json(source_path: str,
                   checkpoint_path: str,
                   target_table_path: str):
    (spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.schemaLocation", checkpoint_path)
        .load(source_path)
        _______
    )
```

Which response correctly fills in the blank to meet the specified requirements?

A.
```
.writeStream
.option("mergeSchema", True)
.start(target_table_path)
```

B.
```
.writeStream
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
.trigger(once=True)
.start(target_table_path)
```

C.
```
.write
.option("checkpointLocation", checkpoint_path)
.option("mergeSchema", True)
.outputMode("append")
.save(target_table_path)
```

D.
```
  .write
  .option("mergeSchema", True)
  .mode("append")
  .save(target_table_path)
```

E.
```
  .writeStream
  .option("checkpointLocation", checkpoint_path)
  .option("mergeSchema", True)
  .start(target_table_path)
```

Answer: E

Explanation:

Correct answer is E, it is a streaming write, and the default outputMode is Append (so if it's optional in this case)

Question: 78 **Deep Dumps**

The data engineering team maintains the following code:

```
import pyspark.sql.functions as F

(spark.table("silver_customer_sales")
  .groupBy("customer_id")
  .agg(
    F.min("sale_date").alias("first_transaction_date"),
    F.max("sale_date").alias("last_transaction_date"),
    F.mean("sale_total").alias("average_sales"),
    F.countDistinct("order_id").alias("total_orders"),
    F.sum("sale_total").alias("lifetime_value")
  ).write
  .mode("overwrite")
  .table("gold_customer_lifetime_sales_summary")
)
```

Assuming that this code produces logically correct results and the data in the source table has been de-duplicated and validated, which statement describes what will occur when this code is executed?

A.The silver_customer_sales table will be overwritten by aggregated values calculated from all records in the gold_customer_lifetime_sales_summary table as a batch job.

B.A batch job will update the gold_customer_lifetime_sales_summary table, replacing only those rows that have different values than the current version of the table, using customer_id as the primary key.

C.The gold_customer_lifetime_sales_summary table will be overwritten by aggregated values calculated from all records in the silver_customer_sales table as a batch job.

D.An incremental job will leverage running information in the state store to update aggregate values in the gold_customer_lifetime_sales_summary table.

E.An incremental job will detect if new rows have been written to the silver_customer_sales table; if new rows are detected, all aggregates will be recalculated and used to overwrite the gold_customer_lifetime_sales_summary table.

Answer: C

Explanation:

The gold_ customer_ lifetime_ sales_ summary table will be overwritten by aggregated values calculated from all records in the silver_ customer_ sales table as a batch job.

Question: 79 **Deep Dumps**

The data architect has mandated that all tables in the Lakehouse should be configured as external (also known as "unmanaged") Delta Lake tables.

Which approach will ensure that this requirement is met?

A.When a database is being created, make sure that the LOCATION keyword is used.

B.When configuring an external data warehouse for all table storage, leverage Databricks for all ELT.

C.When data is saved to a table, make sure that a full file path is specified alongside the Delta format.

D.When tables are created, make sure that the EXTERNAL keyword is used in the CREATE TABLE statement.

E.When the workspace is being configured, make sure that external cloud object storage has been mounted.

Answer: A

Explanation:

A. When a database is being created, make sure that the LOCATION keyword is used.

Why this is correct

In Databricks / Delta Lake:

If a database (schema) is created with a LOCATION, all tables created inside that database are external (unmanaged) by default.

The data lives in user-managed cloud object storage, and Databricks does not control the data lifecycle.

This is the only option that enforces the requirement globally, without relying on developers to remember table-level rules.

Question: 80 **Deep Dumps**

The marketing team is looking to share data in an aggregate table with the sales organization, but the field names used by the teams do not match, and a number of marketing-specific fields have not been approved for the sales

org.

Which of the following solutions addresses the situation while emphasizing simplicity?

A.Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.

B.Create a new table with the required schema and use Delta Lake's DEEP CLONE functionality to sync up changes committed to one table to the corresponding table.

C.Use a CTAS statement to create a derivative table from the marketing table; configure a production job to propagate changes.

D.Add a parallel table write to the current production pipeline, updating a new sales table that varies as required from the marketing table.

E.Instruct the marketing team to download results as a CSV and email them to the sales organization.

Answer: A

Explanation:

The best solution is **A. Create a view on the marketing table selecting only those fields approved for the sales team; alias the names of any fields that should be standardized to the sales naming conventions.**

Here's why:

Simplicity: Views are lightweight and don't involve data duplication or complex data pipelines. They are simply stored queries that present data from the underlying table.
Data Governance/Security: Creating a view allows you to explicitly control which columns are exposed to the sales team, enforcing data governance policies and preventing unauthorized access to sensitive marketing data. The view acts as a filter and restricts data based on the agreed-upon fields.
Schema Transformation: Views can easily alias column names, mapping marketing-specific field names to the sales organization's conventions. This resolves the field name mismatch issue directly within the view definition.
Real-time/Near Real-time access: Changes made to the underlying marketing table are immediately reflected in the view (upon querying it), providing the sales team with up-to-date aggregate data.
Low Overhead: Views consume minimal resources compared to creating and managing separate tables.

The other options are less suitable:

B. DEEP CLONE is usually used to migrate a table from one platform to another.
C. CTAS creates a new table, duplicating data and requiring a pipeline to maintain consistency. This adds complexity and storage overhead.
D. Parallel table write also creates a new table, requiring changes to an existing production pipeline. This is more complex than creating a view.
E. CSV and email is a manual process and unsuitable for regular data sharing and collaboration. It also has data security and compliance implications.

Authoritative Links:

Databricks Views: https://docs.databricks.com/sql/language-manual/sql-ref-syntax-ddl-create-view.html
Delta Lake DEEP CLONE: https://docs.databricks.com/delta/clone.html

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

But Wait

I wanted to let you know that there is more content available in the full version.
The full paper contains additional sections and information that you may find helpful, and I encourage you to download it to get a more comprehensive and detailed view of all the subject matter.

Total: **369 Questions**

Link: https://deepdumps.com/papers/databricks/certified-data-engineer-professional